KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

INDUSTRY PROJECT REPORT

CAT-AVO

Continuous Adaptive Trust & Automated Vulnerability Orchestration

Post-authentication behavioural trust using an unsupervised PyTorch autoencoder in an isolated VMware lab

Submitted by

Yash Sahu

Roll No. 25168014

November 2026

Abstract

Authentication protects the front door, but most identity systems trust a session once it has been opened. CAT-AVO (Continuous Adaptive Trust & Automated Vulnerability Orchestration) investigates post-authentication behavioural trust: scoring how a session behaves and reacting automatically when it drifts from normal.

The project builds an isolated VMware lab of seven machines: a pfSense firewall, a Windows Server 2019 domain controller, Keycloak as identity provider federated to Active Directory, Splunk Enterprise as SIEM (sharing one RHEL virtual machine with Keycloak), a Windows 11 endpoint, a Kali attacker and the host laptop running the SOAR and machine-learning code. Authentication and endpoint events are centralised in Splunk.

A PyTorch autoencoder (4-3-2-3-4) is trained only on 720 normal logins using four features: session duration, failed attempts before login, new source address and the gap between logins. Its reconstruction error, thresholded at the 99th percentile of training error, drives an automated response that revokes the Keycloak session and blocks the source on the RHEL firewall and on pfSense.

Evaluated on 54 simulated attack runs, the autoencoder detected 83.3 % with a hold-out false-positive rate of 2.4 %, against 70.4 % for Splunk correlation rules and 59.3 % for an Isolation Forest at a looser threshold. The average time from event to completed response was 7.2 seconds, dominated by SIEM indexing and polling. Limitations (small synthetic evaluation, two users, undetected stolen-credential attacks from a known address) and future work (Active Directory lockout, vulnerability scanning) are discussed.

Keywords: continuous adaptive trust, autoencoder, anomaly detection, Keycloak, Splunk, SOAR, pfSense, identity security

Index

1. Introduction 1
1.1 Background 1
1.2 Problem statement 1
1.3 Objectives 1
1.4 Scope 1
1.5 Report organisation 1
2. Approach and Novelty 3
3. System Architecture 4
3.1 Overall design 4
3.2 Machines and roles 5
3.3 Software stack 5
4. Lab Implementation 6
4.1 pfSense firewall 6
4.2 Active Directory domain controller 9
4.3 RHEL host: Keycloak and Splunk 10
4.4 Keycloak identity provider 11
4.5 Splunk Enterprise 14
4.6 Windows 11 endpoint 18
4.7 Kali Linux attacker 21
4.8 Resource footprint 22
5. Commands and Verified Outputs 23
5.1 Laptop: project environment (Windows PowerShell) 23
5.2 Laptop: building the dataset and training the models 25
5.3 Windows 11 virtual machine (CATAVO-WS01): normal-behaviour generator 27
5.4 Windows Server 2019 (CATAVO-AD): domain controller 29
5.5 RHEL 10.2 server (catavo-keycloak): network and Splunk start 29
5.6 Kali Linux: attack commands 31
5.7 Laptop: response engine and monitor 31
6. Telemetry Pipeline 33
7. Data Collection and Feature Engineering 36
7.1 Normal-behaviour data 36
7.2 Features 37
7.3 Dataset split 37
8. Machine-Learning Model 38
8.1 Autoencoder 38
8.2 Baselines 38
8.3 Program files 38
9. Automated Response (SOAR) 40
10. Attack Simulation 41
11. Source Code Explained 42
11.1 features.py (first version, superseded) 42
11.2 features2.py (final data preparation) 43
11.3 catavo_ml.py (the autoencoder) 43

11.4 baseline_if.py (Isolation Forest baseline) 44
11.5 soar.py (automated response) 44
11.6 monitor.py (live detection loop) 45
11.7 attacks.py (attack simulator) 45
11.8 evaluate.py (evaluation) 46
12. Evaluation and Results 47
12.1 Detection performance 47
12.2 Per-scenario results 48
12.3 Feature ablation 48
12.4 Effect of removing the hour of day 49
13. Response Time 50
14. Findings 51
15. Limitations 52
16. Future Work 53
17. Conclusion 54
Appendix A: Source Code Listings 54
features.py: first feature builder (superseded) 55
features2.py: final data preparation 56
catavo_ml.py: autoencoder training and scoring 58
baseline_if.py: Isolation Forest baseline 60
soar.py: automated response 61
monitor.py: live detection loop 63
attacks.py: attack simulator 66
evaluate.py: evaluation 68
Appendix B: Complete Evaluation Output 71
Appendix C: Splunk Searches Used 73

1. Introduction

1.1 Background

Most enterprise security effort is spent on the moment of login: passwords, multi-factor authentication and single sign-on. Once a user is authenticated, however, the session is usually trusted until it expires. A stolen password, a hijacked session or a malicious insider then behaves like a legitimate user, and conventional controls stay silent because every individual request is well formed and correctly authenticated.

Continuous adaptive trust addresses this gap. Instead of treating authentication as a single gate, the system keeps scoring how each session behaves against what is normal for that user and reacts as soon as the behaviour drifts. CAT-AVO (Continuous Adaptive Trust & Automated Vulnerability Orchestration) is an Industry Project that builds and evaluates this idea end to end in an isolated virtual lab.

1.2 Problem statement

Rule-based detection (for example "five failed logins in five minutes") is easy to write but easy to evade, and it cannot describe what is normal for a particular user. Manually reacting to alerts is slow. The project therefore asks: can an unsupervised model, trained only on normal login behaviour, recognise abnormal post-authentication activity in near real time, and can the alert automatically trigger containment across the identity provider and the network?

1.3 Objectives

- Build an isolated, reproducible lab with a firewall, a directory service, an identity provider, a SIEM, an endpoint and an attacker machine.
- Collect authentication and endpoint telemetry centrally in Splunk Enterprise.
- Train an unsupervised PyTorch autoencoder on normal behaviour only and use its reconstruction error as a trust signal.
- Automate the response (SOAR): revoke the Keycloak session and block the offending source on the host firewall and on pfSense.
- Evaluate the system with repeatable attack simulations against baselines (Splunk correlation rules and Isolation Forest), and report detection, false positives and response time honestly.

1.4 Scope

The work covers the complete pipeline from login event to automated response in a lab of seven machines. Two users (user1 and user2) are modelled. Attacks are simulated by scripts written for this project. Vulnerability scanning and Active Directory account lockout are outside the implemented scope and are listed as future work (Chapter 16).

1.5 Report organisation

Chapter 2 positions the approach and Chapter 3 describes the architecture. Chapter 4 documents how each virtual machine was built, and Chapter 5 lists the commands that were run on every machine

together with their verified output and an explanation. Chapter 6 covers the telemetry pipeline, Chapter 7 the data and features and Chapter 8 the model. Chapter 9 describes the response engine, Chapter 10 the attack simulation and Chapter 11 explains every source-code file; the full listings are in Appendix A. Chapters 12 to 14 present the results, the response time and the findings. Limitations, future work and the conclusion close the report.

2. Approach and Novelty

Existing lab-scale demonstrations of identity security typically stop at detection rules or at an offline model evaluated on a public dataset. CAT-AVO differs in three ways.

- **Post-authentication focus.** The model scores sessions after a successful or failed authentication using identity-provider events, not network packets.
- **Unsupervised and per-environment.** The autoencoder is trained only on the normal behaviour of this lab, so no labelled attack data are needed for training.
- **Closed loop.** A detection is not just an alert: the same pipeline revokes the session and blocks the source, and the delay of that loop is measured.

Two baselines are used for a fair comparison: a rule-based detector written as Splunk correlation logic, and an Isolation Forest trained on the same features, scaling and split as the autoencoder.

3. System Architecture

3.1 Overall design

The lab runs in VMware Workstation on a laptop with 32 GB of RAM. All virtual machines are attached to a single isolated host-only segment, VMnet2 (192.168.100.0/24), behind a pfSense firewall. The laptop itself also sits on the segment and hosts the SOAR and machine-learning code. Figure 1 shows the architecture and Figure 2 the intended working flow.

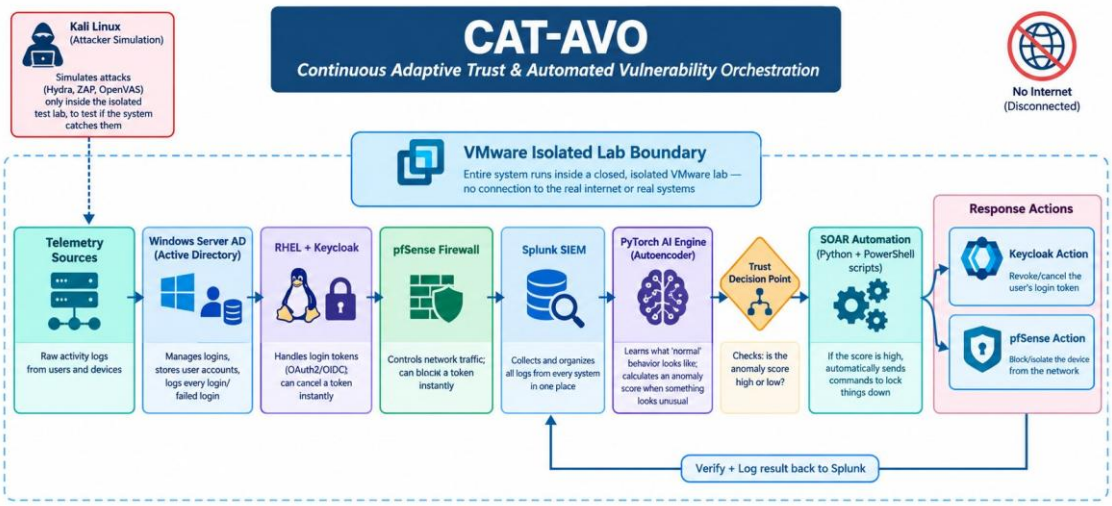


Figure 1: System architecture of the CAT-AVO lab

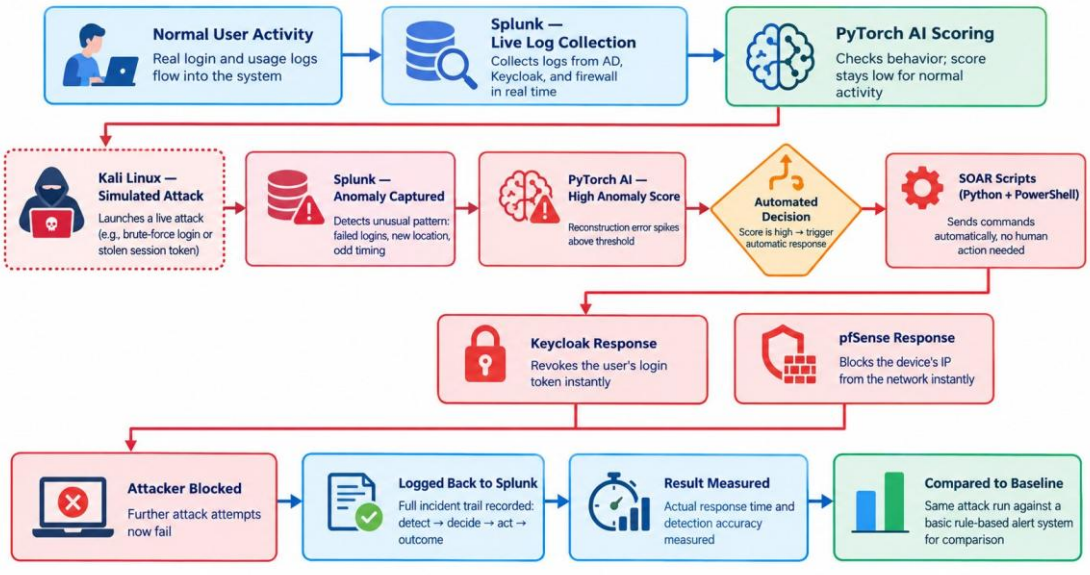


Figure 2: End-to-end working flow of CAT-AVO

3.2 Machines and roles

Table 1: Lab machines

Machine	Address	Operating system	Role
pfSense	192.168.100.1	pfSense 2.4.5-p1	Firewall, DHCP, syslog source, SOAR block list
CATAVO-AD	192.168.100.10	Windows Server 2019 Standard Evaluation (Desktop Experience)	Active Directory domain controller, domain catavo.local
catavo-keycloak	192.168.100.20	Red Hat Enterprise Linux 10.2	Keycloak 26.8.0 identity provider (port 8080)
catavo-keycloak (same VM)	192.168.100.30	Red Hat Enterprise Linux 10.2	Splunk Enterprise 10.6.0.5: web 8000, REST 8089, receiving 9997, syslog UDP 5514
Kali	192.168.100.40	Kali Linux	Attacker machine (static address, no gateway, no Internet)
CATAVO-WS01	192.168.100.50	Windows 11 Pro	Domain-joined endpoint with Splunk Universal Forwarder; baseline machine of user1
Laptop	192.168.100.70	Host operating system	SOAR and ML code; also plays user2 (baseline) and a second attack source

Lab simplification. To fit the memory budget of one laptop, Keycloak and Splunk Enterprise share a single RHEL virtual machine that carries two addresses (192.168.100.20 for Keycloak and 192.168.100.30 for Splunk). They are logically separate services and are addressed separately throughout the lab; in production they would run on separate hosts.

3.3 Software stack

Table 2: Software stack

Layer	Component	Purpose
Firewall	pfSense 2.4.5-p1	Segmentation, DHCP, syslog, block list
Directory	Active Directory (Windows Server 2019)	Authoritative user store
Identity provider	Keycloak 26.8.0, realm catavo	Authentication, session management, event log
SIEM	Splunk Enterprise 10.6.0.5 and Universal Forwarder	Central log collection and search
Machine learning	Python, PyTorch, scikit-learn, pandas	Autoencoder and Isolation Forest
Response	Python SOAR module, firewalld, pfSense pf tables	Containment actions
Attack tools	Kali Linux and Python attack scripts	Simulated attacks

All machines use Indian Standard Time so that timestamps from Keycloak, Splunk, Windows and the monitor can be compared directly.

4. Lab Implementation

4.1 pfSense firewall

pfSense was installed first because every other machine depends on it for addressing and logging. The WAN interface uses NAT for initial updates; the LAN interface was set to the static address 192.168.100.1/24 and the DHCP server was configured for the range 192.168.100.100 to 192.168.100.200, leaving the low addresses for static servers.

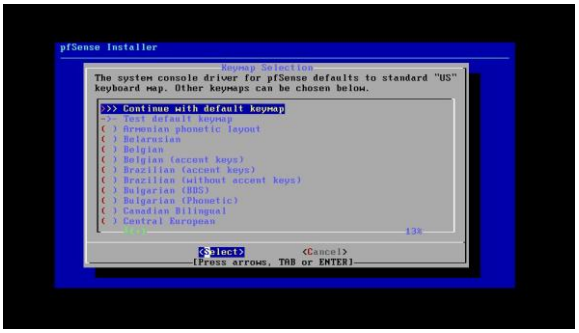


Figure 3: pfSense installer: keyboard map selection



Figure 4: pfSense installation finished

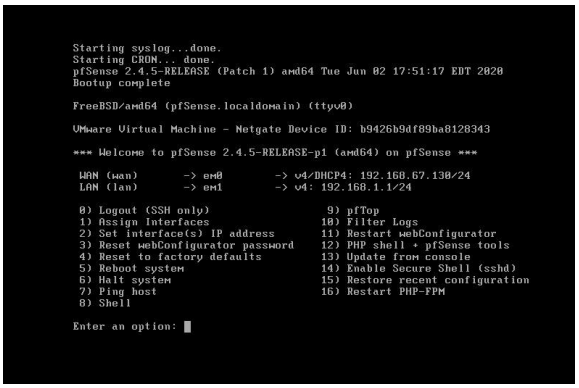


Figure 5: pfSense console menu at first boot

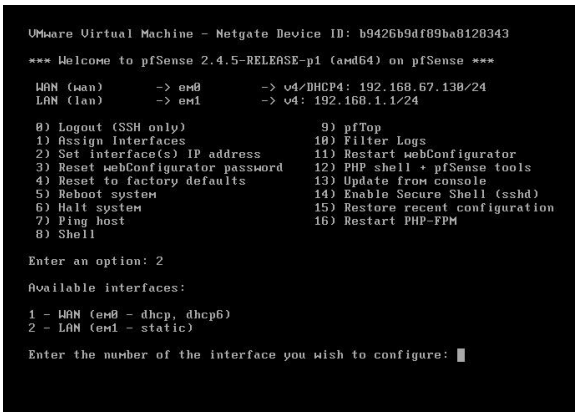


Figure 6: Assigning interfaces from the console menu

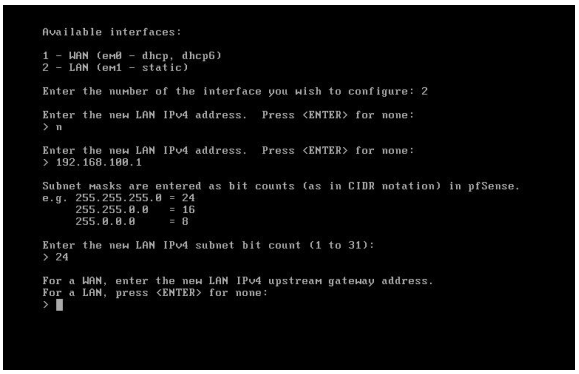
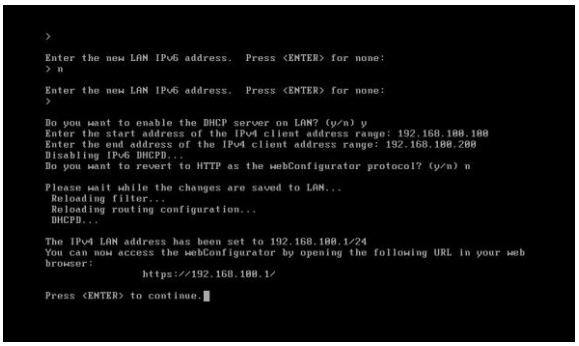


Figure 7: LAN interface set to 192.168.100.1/24



Enter the new LAN IPv6 address. Press <ENTER> for none:
> n

Enter the new LAN IPv6 address. Press <ENTER> for none:
>

Do you want to enable the DHCP server on LAN? (y/n) y
Enter the start address of the IPv4 client address range: 192.168.100.100
Enter the end address of the IPv4 client address range: 192.168.100.200
Disabling IPv6 DHCPD...
Do you want to revert to HTTP as the webConfigurator protocol? (y/n) n

Please wait while the changes are saved to LAN...
Reloading filter...
Reloading routing configuration...
DHCPD...

The IPv4 LAN address has been set to 192.168.100.1/24
You can now access the webConfigurator by opening the following URL in your web browser:
https://192.168.100.1/

Press <ENTER> to continue.

Figure 8: DHCP range 192.168.100.100 to 192.168.100.200

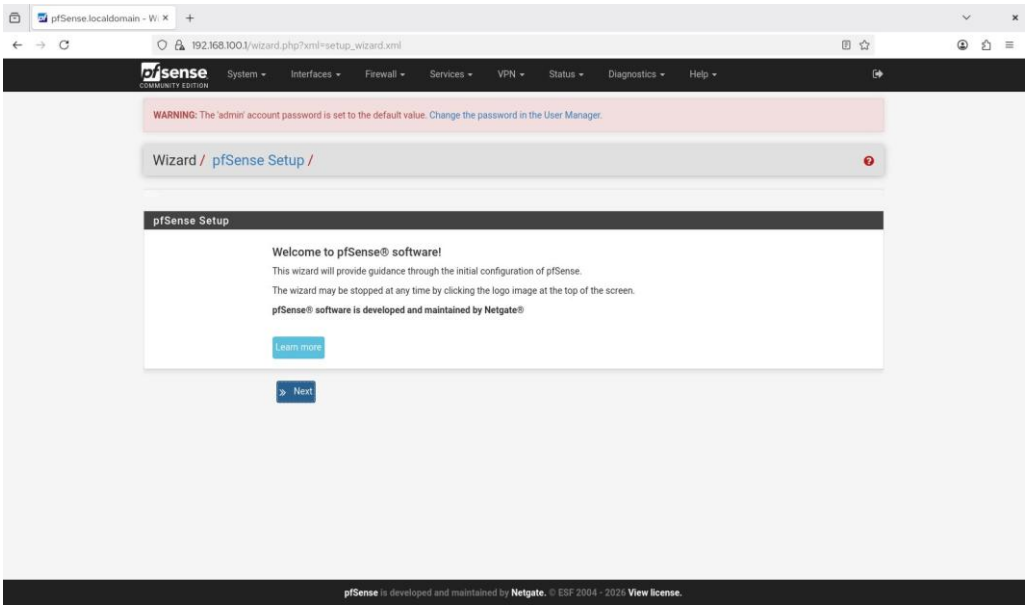


Figure 9: pfSense setup wizard

Two features of pfSense are central to the project. First, it forwards its logs to Splunk by remote syslog (UDP 5514, everything). Second, a firewall alias named blocked_ips is referenced by a block rule at the top of the LAN rules, labelled "SOAR block list". The SOAR engine adds an address to this alias with a single command, and the rule drops its traffic. SSH access was enabled so that the engine can run this command.

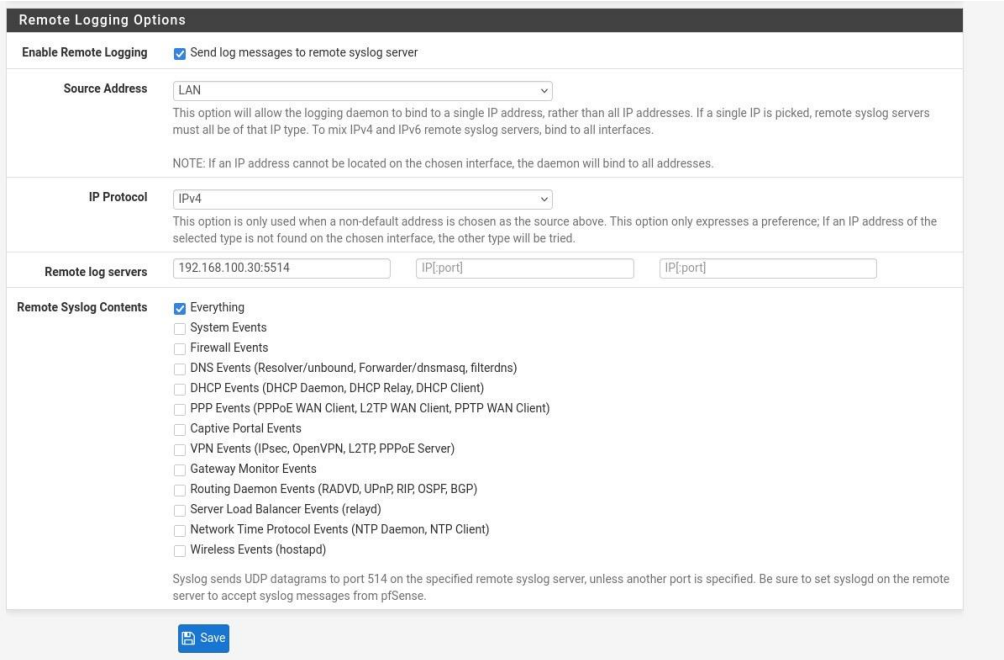


Figure 10: Remote logging from pfSense to Splunk (192.168.100.30, port 5514)

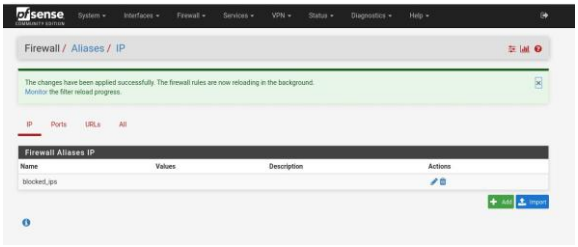


Figure 11: Firewall alias blocked_ips used by the SOAR engine

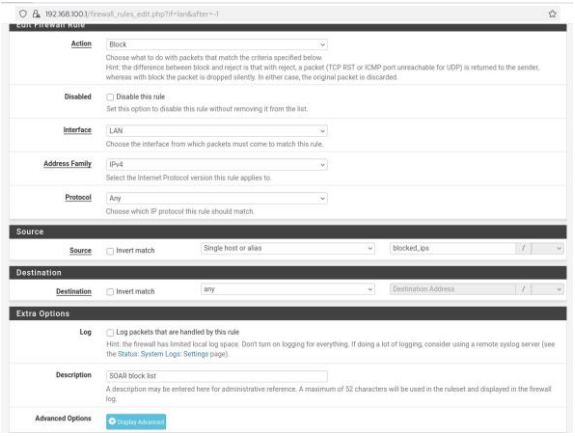


Figure 12: Block rule using the blocked_ips alias as source

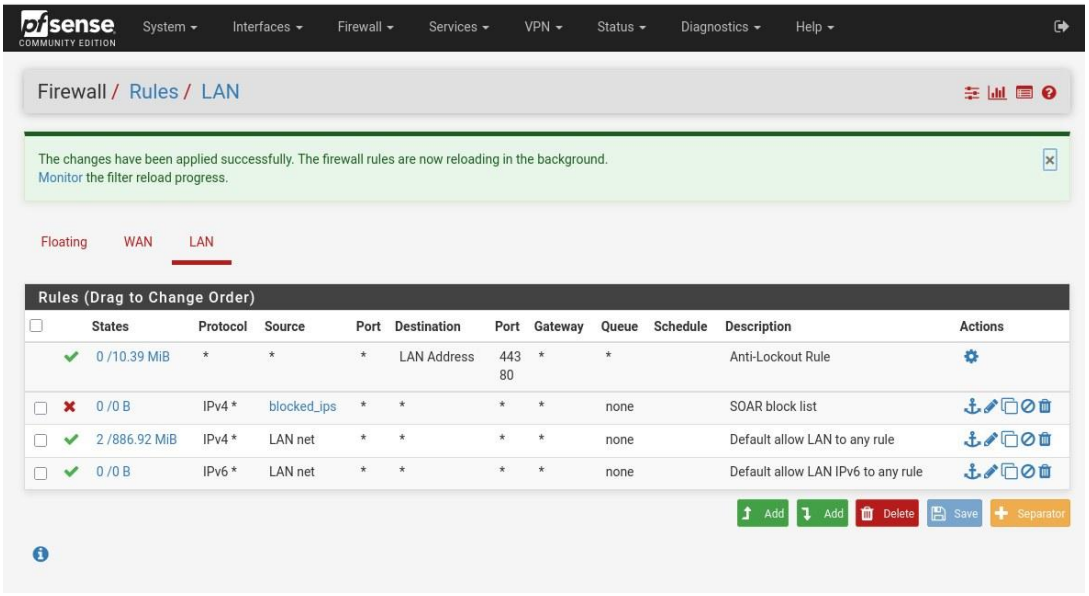


Figure 13: LAN rules with the SOAR block list rule on top

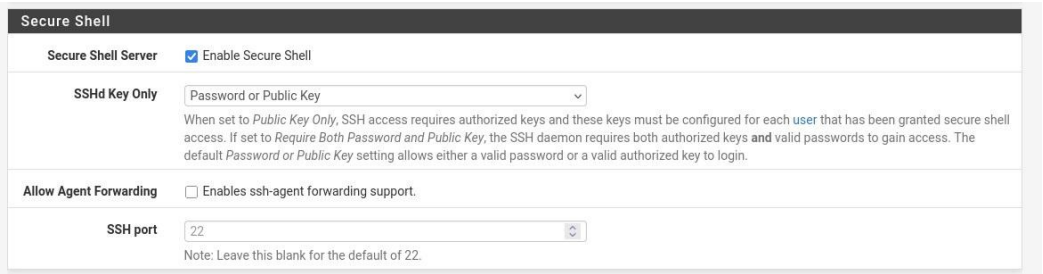


Figure 14: Secure Shell enabled on pfSense for SOAR commands

4.2 Active Directory domain controller

The domain controller CATAVO-AD runs Windows Server 2019 Standard Evaluation with the Desktop Experience. It hosts the domain catavo.local and the user accounts that are later federated into Keycloak. A Splunk Universal Forwarder on the domain controller sends its Application, Security and System logs to Splunk.

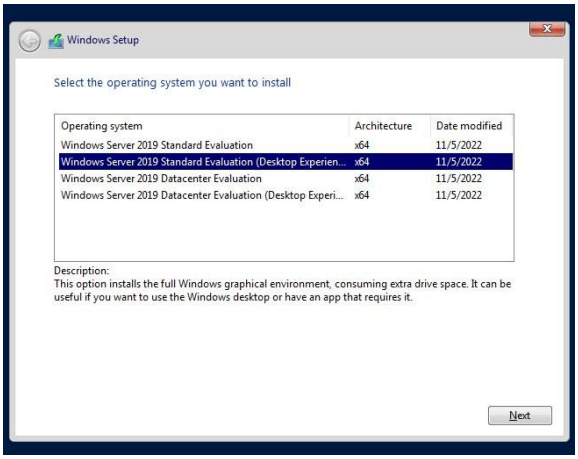


Figure 15: Windows Server 2019 setup: edition selection

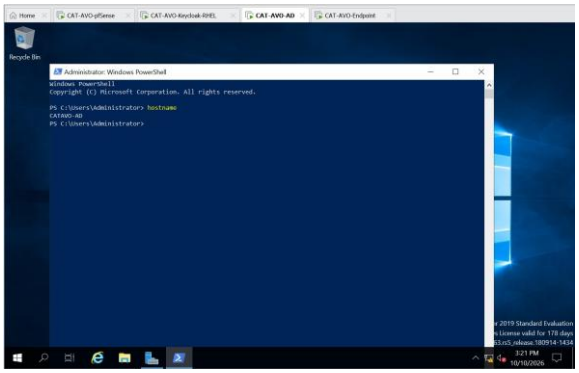


Figure 16: Domain controller CATAVO-AD (Windows Server 2019 Standard Evaluation)

4.3 RHEL host: Keycloak and Splunk

The Red Hat Enterprise Linux 10.2 virtual machine carries both the identity provider and the SIEM. It was given the static address 192.168.100.20 on the lab network; because the lab is isolated by design, it can reach the gateway but not the Internet. Software was therefore installed from locally transferred packages.

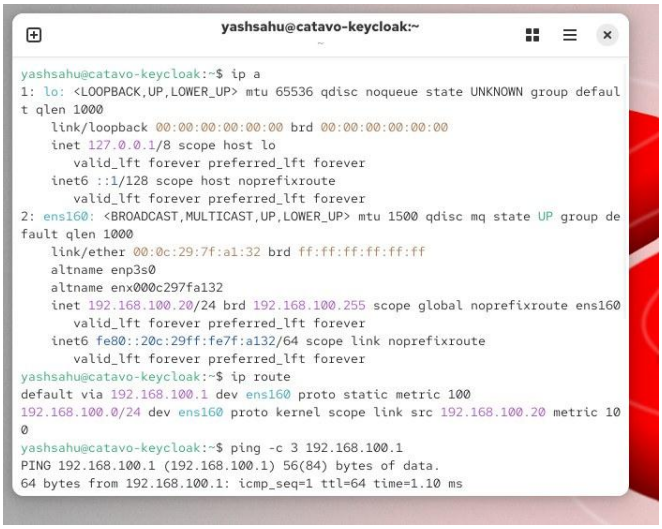


Figure 17: RHEL address and route (192.168.100.20)

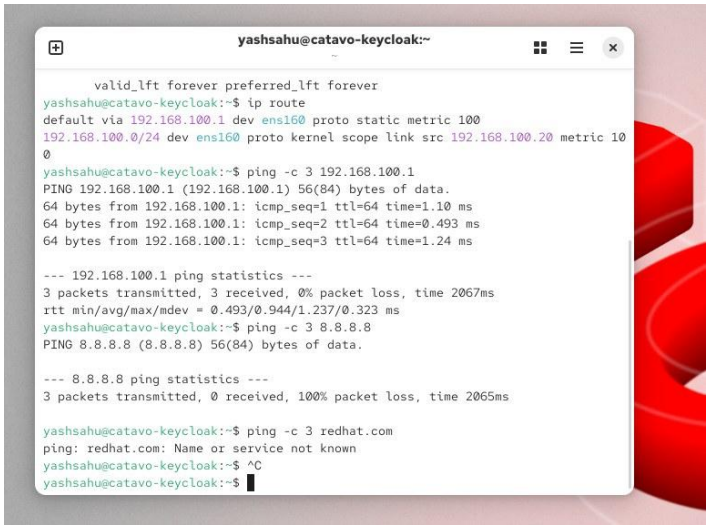


Figure 18: RHEL reaches the gateway; internet is intentionally unreachable

4.4 Keycloak identity provider

Keycloak 26.8.0 listens on port 8080. A dedicated administrator (labadmin) manages the master realm, and a separate realm named catavo holds the lab identities. The realm is connected to Active Directory through an LDAP user federation named ad-catavo, configured read-only; a synchronisation imported the three directory users, including user1 and user2.

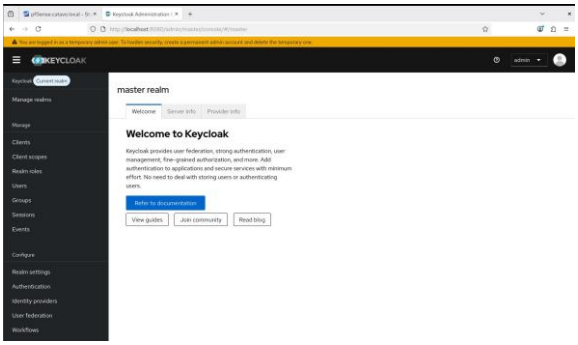


Figure 19: Keycloak first start (master realm)

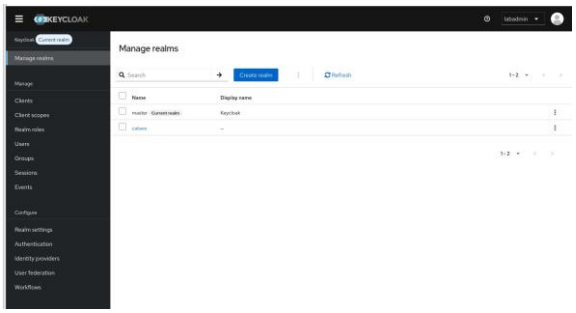


Figure 20: Realms: master and catavo, administrator labadmin

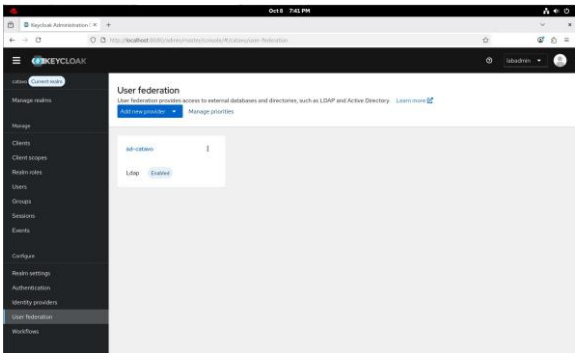


Figure 21: LDAP user federation ad-catavo to Active Directory

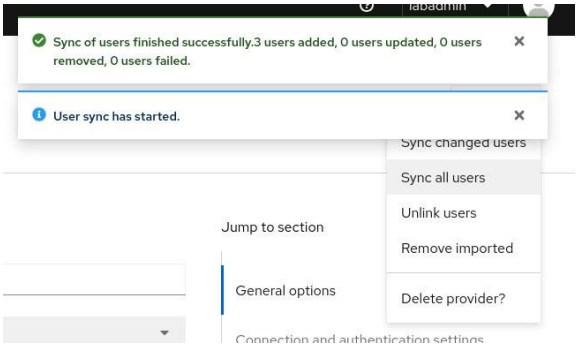


Figure 22: Synchronisation: three users imported from AD

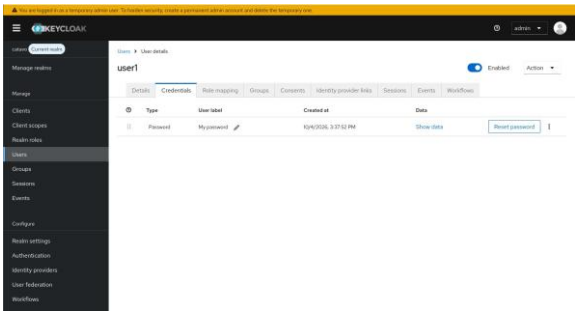


Figure 23: Credentials tab of user1 in realm catavo

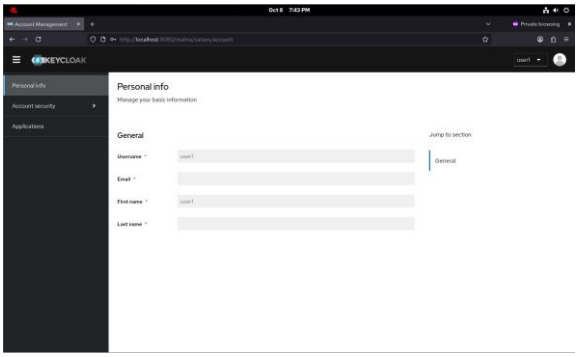


Figure 24: user1 signs in to the Keycloak account console with an AD identity

For the experiments a public OpenID Connect client named catavo-sim was created with the direct access grant enabled. This lets the data generator and the attack scripts perform scripted logins and token requests through the standard token endpoint, producing exactly the LOGIN, LOGIN_ERROR, LOGOUT and CODE_TO_TOKEN events that a real application would produce.

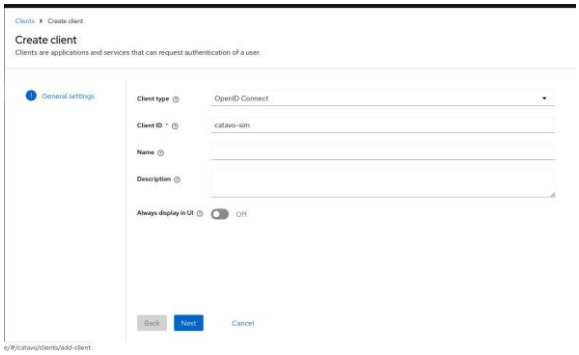


Figure 25: Creating client catavo-sim

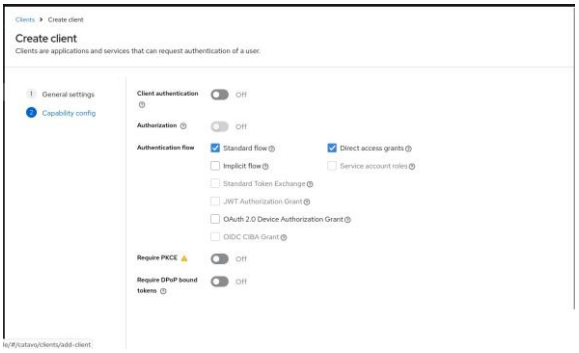


Figure 26: Capability config: direct access grants enabled

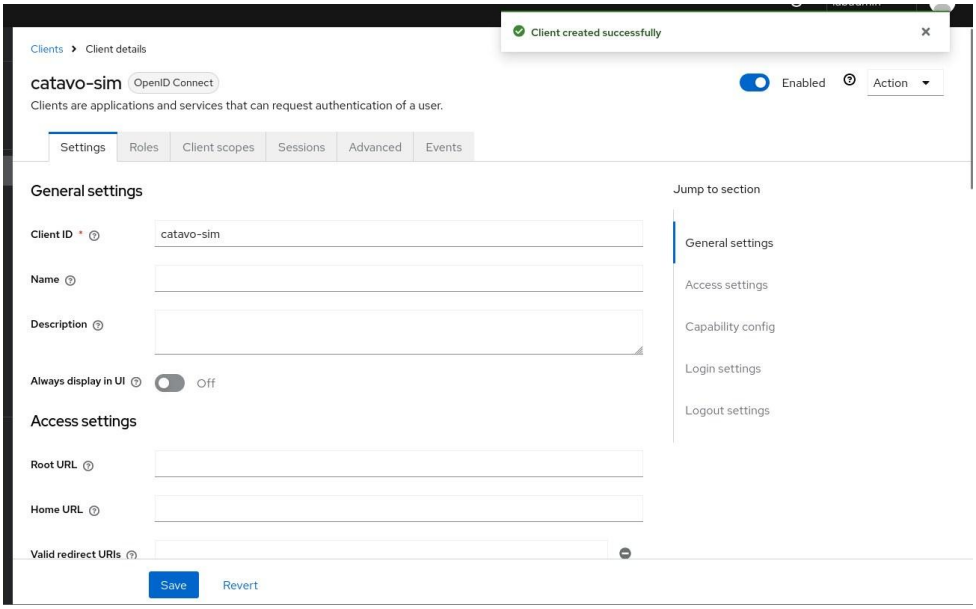


Figure 27: Client catavo-sim created

Event recording is enabled in the realm (user events saved) and the jboss-logging event listener is active, so each authentication event is written to the Keycloak server log with the event type, the user identifier, the session identifier, the client and the source IP address. Splunk reads these records into its keycloak index.

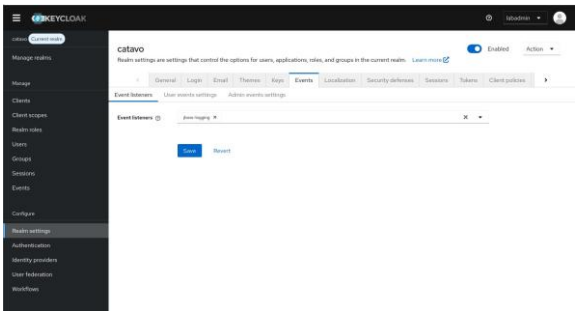


Figure 28: Realm event listener jboss-logging

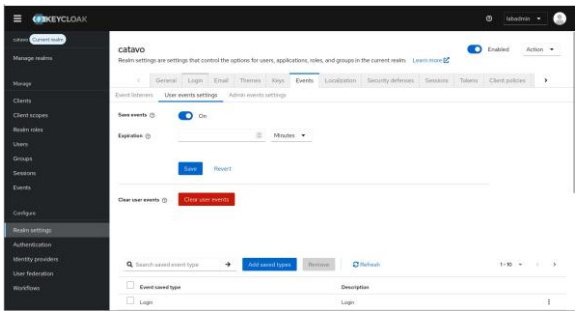


Figure 29: User events saving enabled

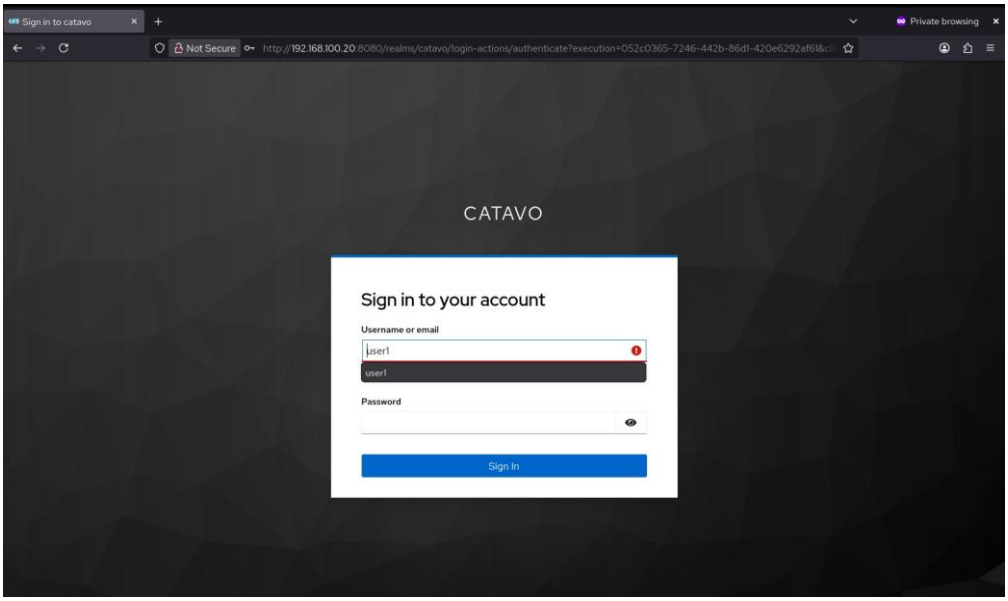


Figure 30: Keycloak sign-in page of realm catavo

4.5 Splunk Enterprise

Splunk Enterprise 10.6.0.5 was started on the RHEL host and four indexes were created, each limited to 5 GB: pfsense (firewall syslog), endpoint (Windows 11 workstation), ad (domain controller) and keycloak (identity events).

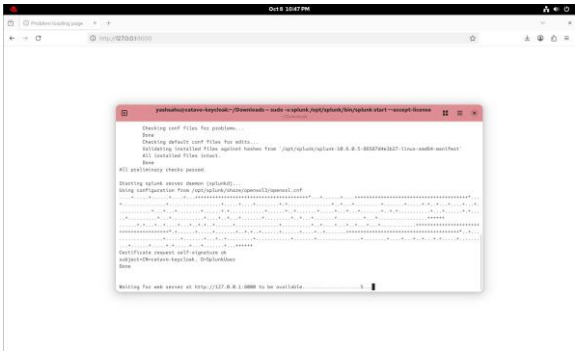


Figure 31: Starting Splunk Enterprise for the first time

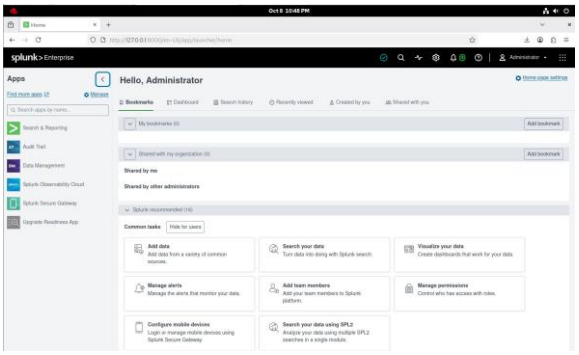


Figure 32: Splunk Enterprise home page

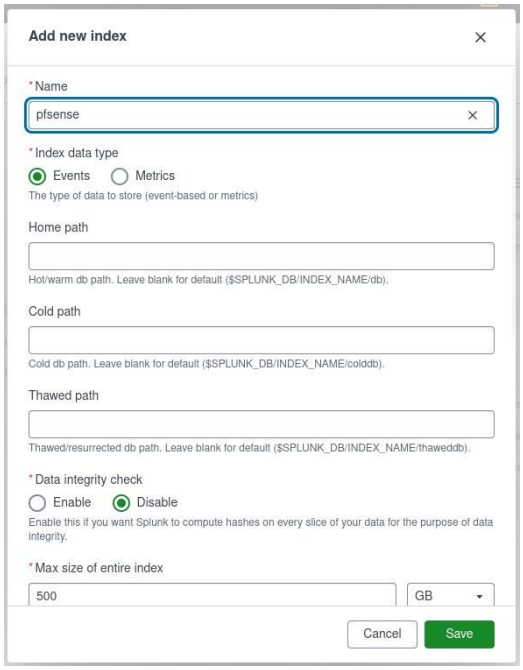


Figure 33: Creating the index pfsense

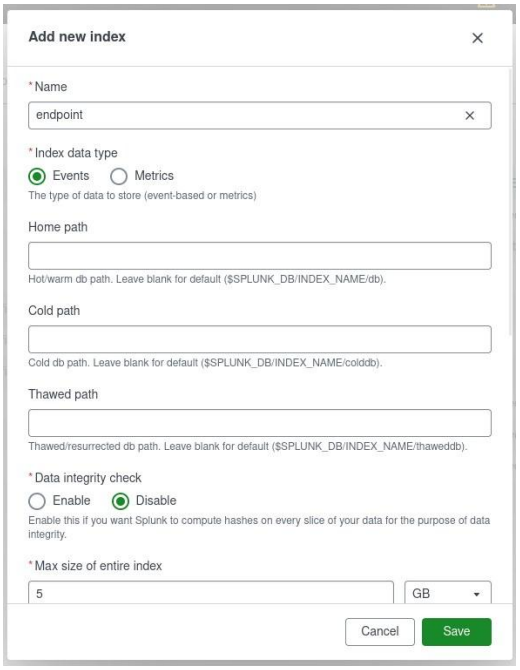


Figure 34: Creating the index endpoint

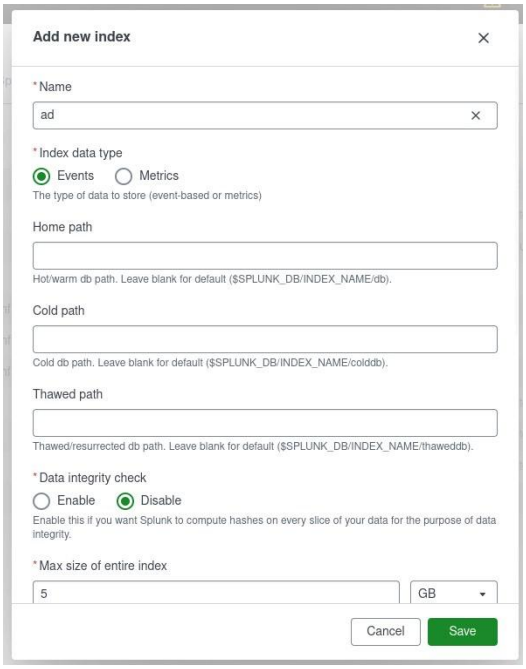


Figure 35: Creating the index ad

Figure 36: Index pfsense created (5 GB maximum)

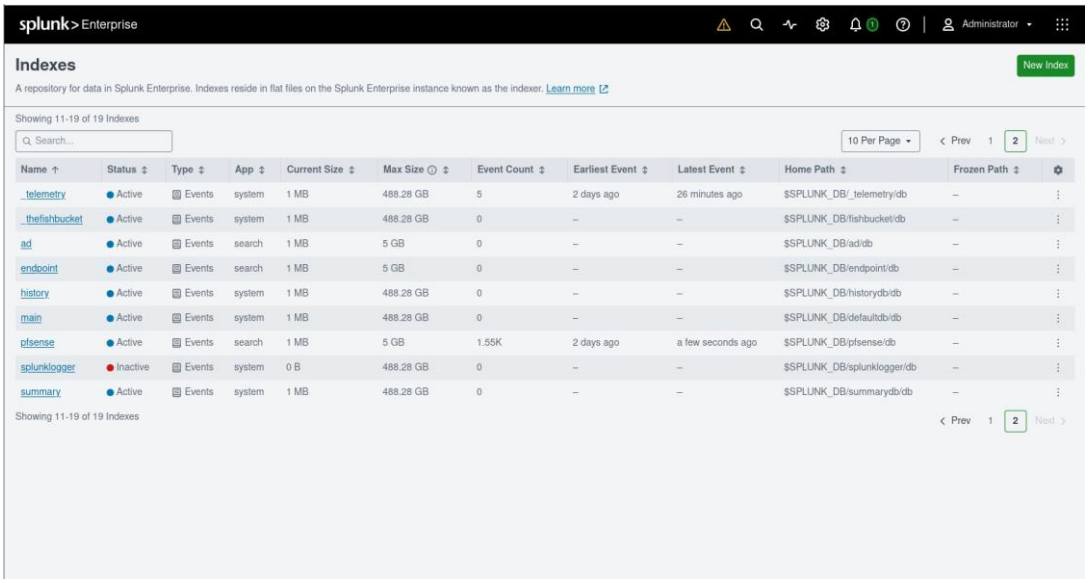


Figure 37: Splunk index list showing ad, endpoint and pfsense

pfSense logs arrive through a UDP data input on port 5514 with the source type syslog, restricted to the host 192.168.100.1 and stored in the pfsense index. A first search confirmed that firewall events were being indexed.

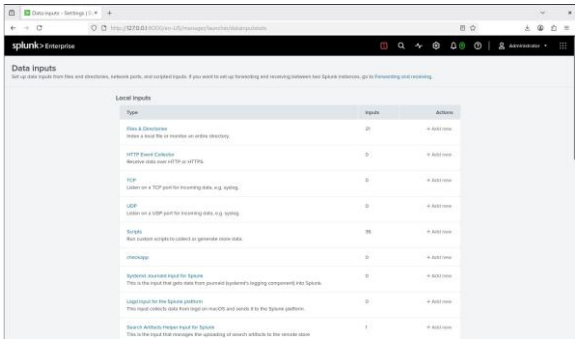


Figure 38: Splunk data inputs

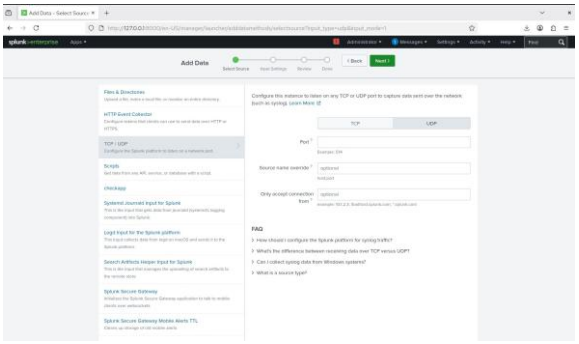


Figure 39: Add data: select UDP source

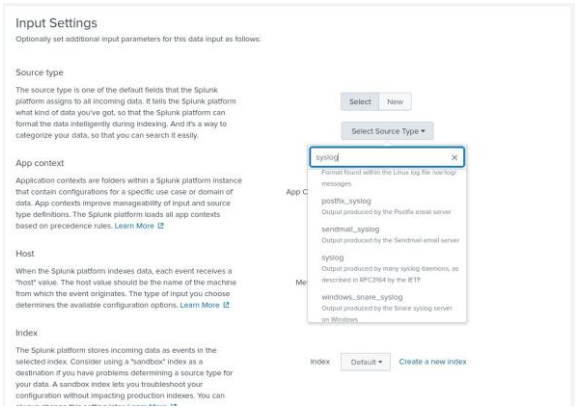


Figure 40: Input settings: source type syslog

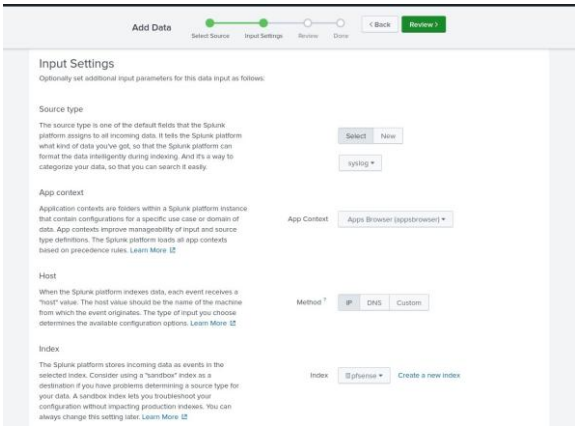


Figure 41: Input settings: index pfsense

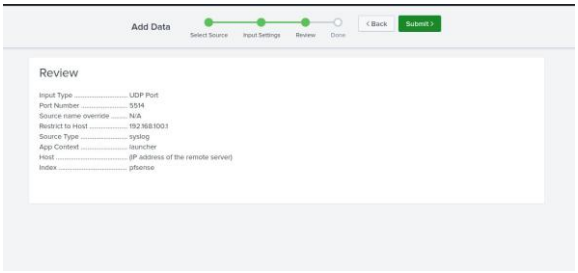


Figure 42: Review: UDP 5514 restricted to 192.168.100.1

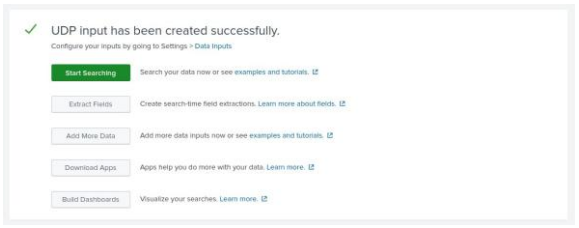


Figure 43: UDP input created successfully

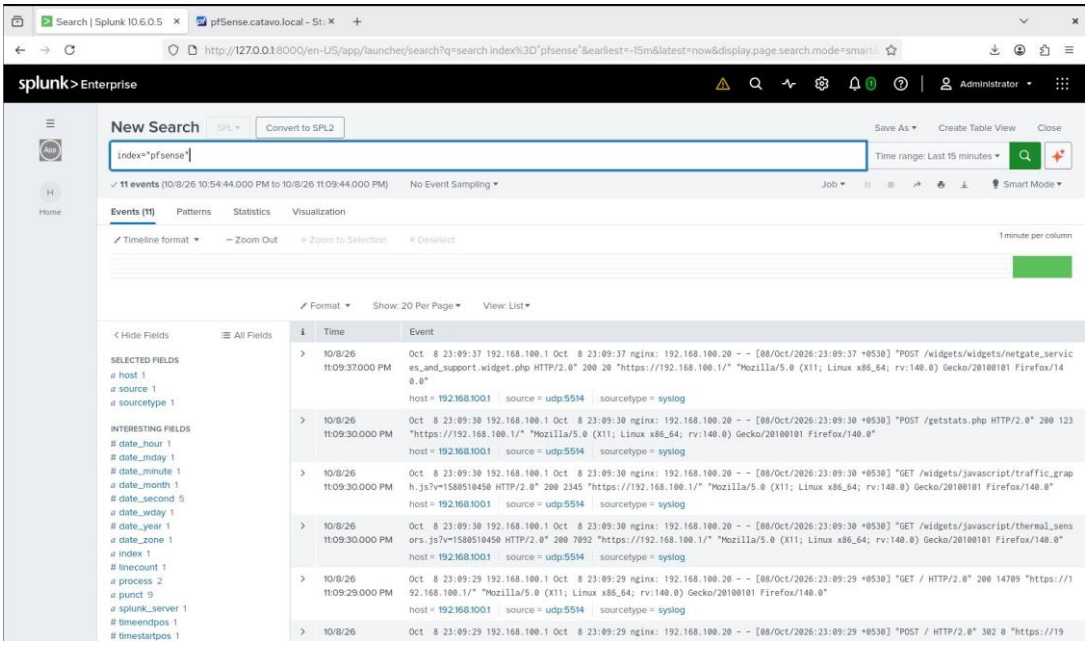


Figure 44: First pfSense events visible in Splunk

For the Windows machines Splunk receives data from Universal Forwarders on TCP port 9997.

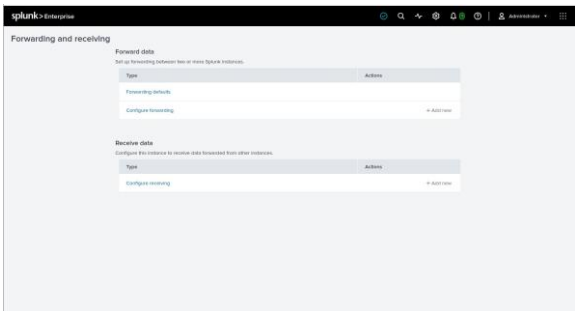


Figure 45: Forwarding and receiving page

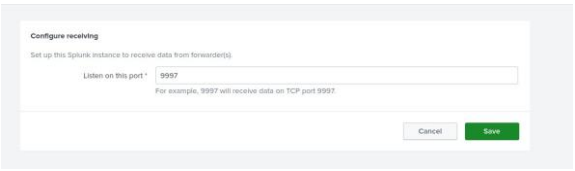


Figure 46: Receiving enabled on port 9997

4.6 Windows 11 endpoint

CATAVO-WS01 is a Windows 11 Pro virtual machine joined to the domain catavo.local. It represents the normal workstation of user1; domain users such as user1@catavo.local can sign in on it. The Splunk Universal Forwarder installed on it points to the receiving indexer 192.168.100.30:9997 and forwards the Application, Security and System event logs into the endpoint index.

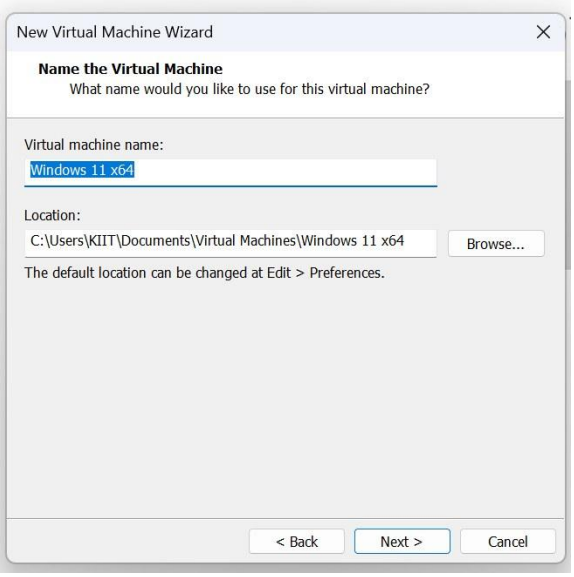


Figure 47: Creating the Windows 11 virtual machine

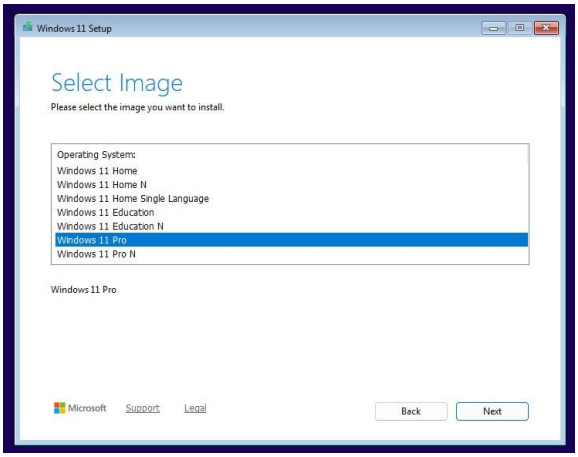


Figure 48: Windows 11 setup: Pro edition



Figure 49: Windows 11 installation in progress



Figure 50: Windows 11 desktop

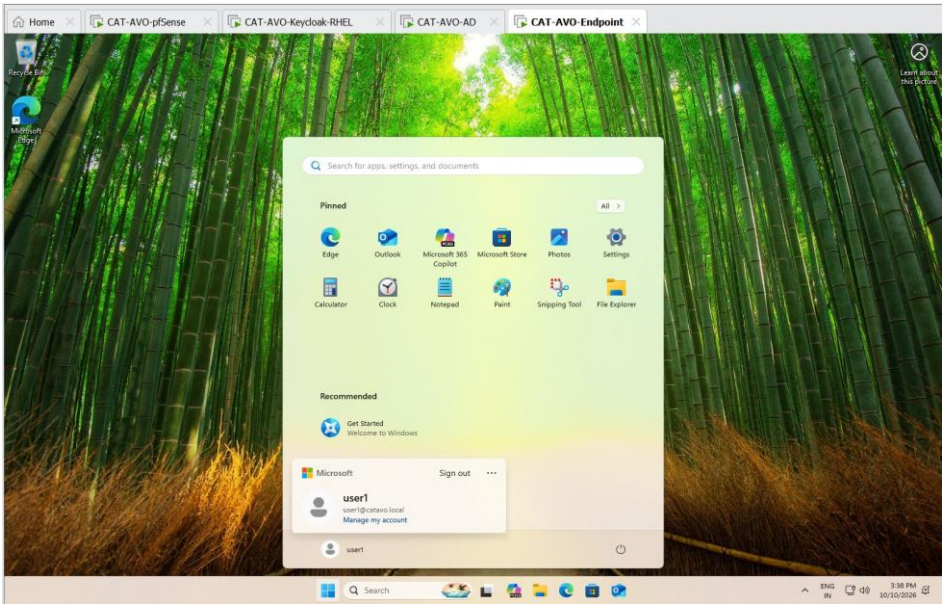


Figure 51: Domain user user1@catavo.local signed in on CATAVO-WS01

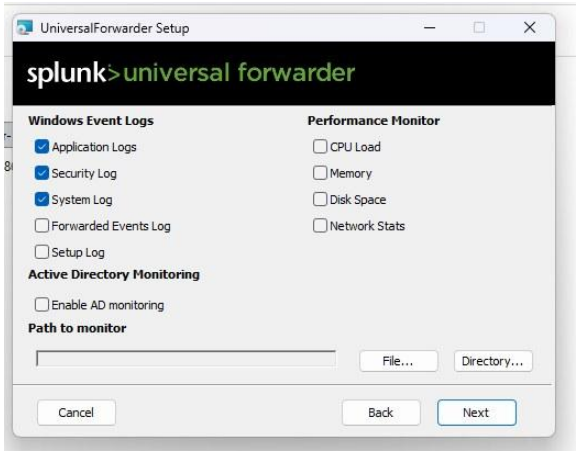


Figure 52: Universal Forwarder: Application, Security and System logs selected

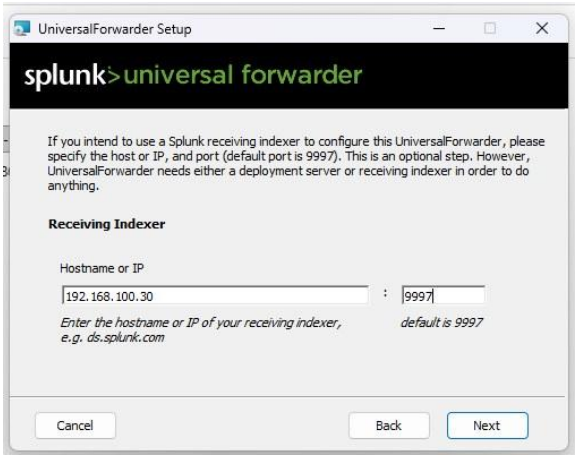


Figure 53: Universal Forwarder: receiving indexer 192.168.100.30:9997

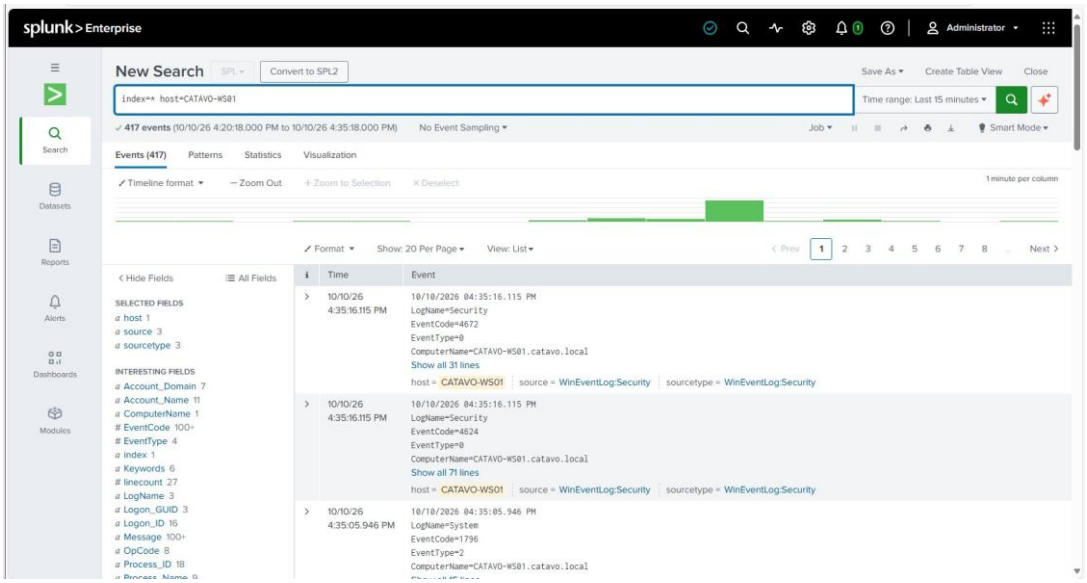


Figure 54: Windows events from CATAVO-WS01 in Splunk

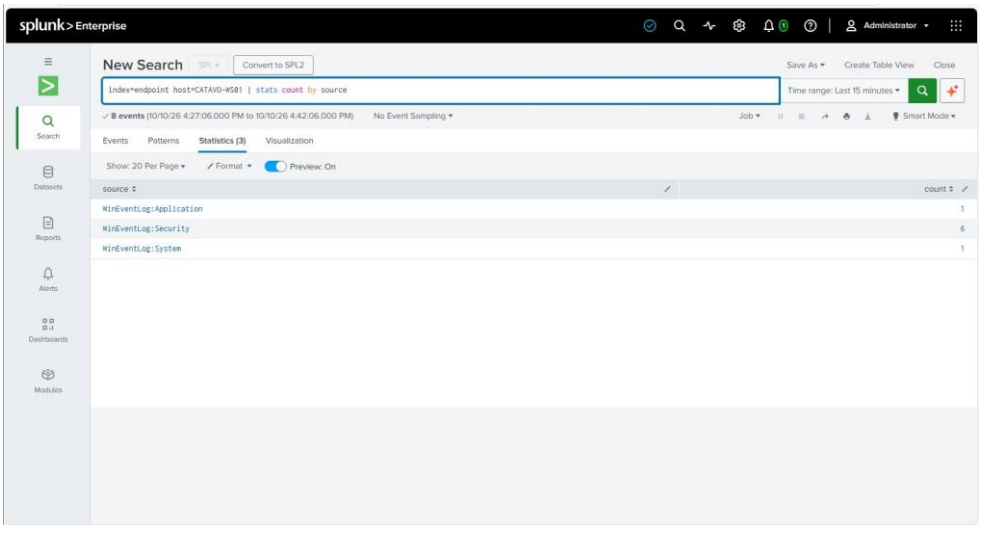


Figure 55: Three Windows log sources received from the workstation

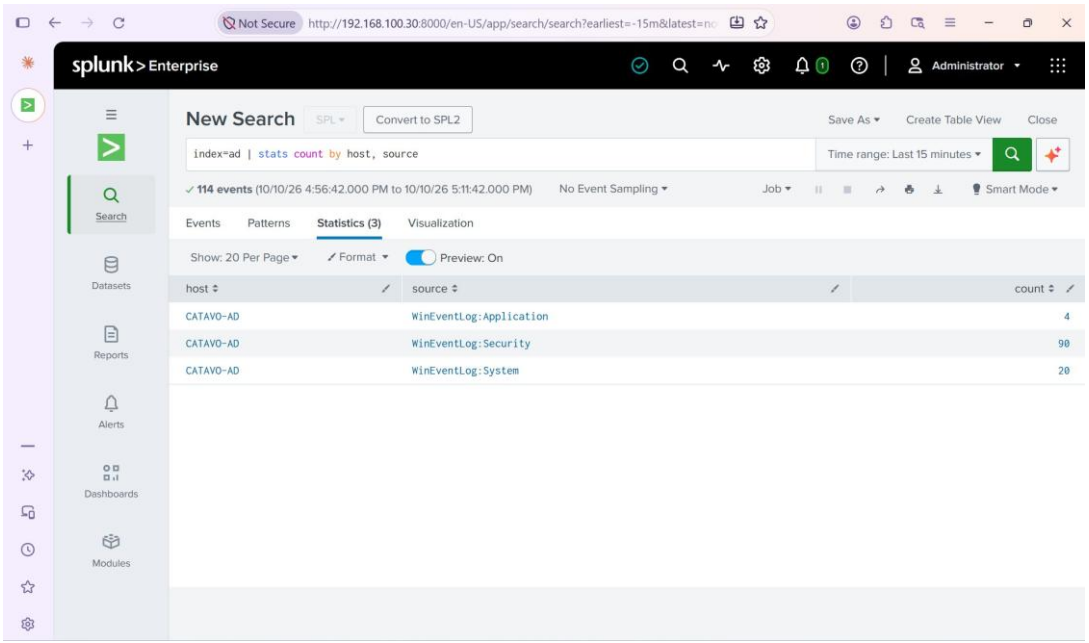


Figure 56: Domain controller events received in Splunk

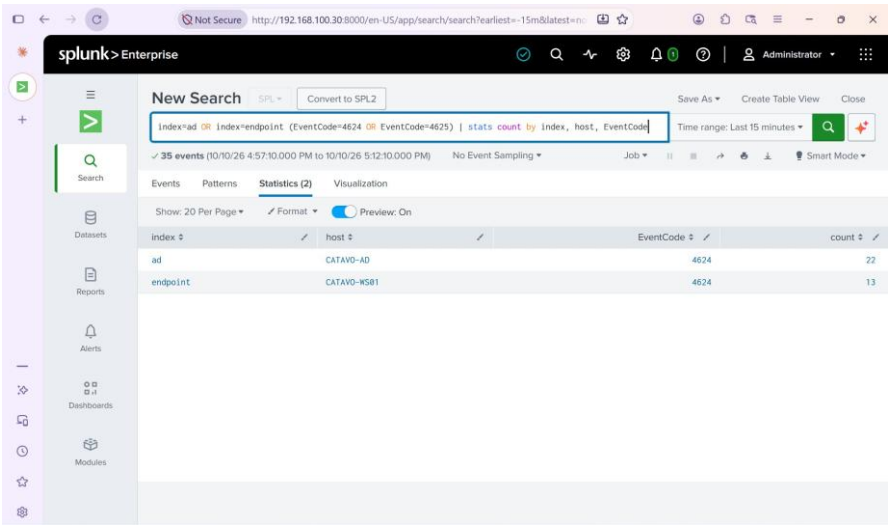


Figure 57: Logon events (4624) per index and host

4.7 Kali Linux attacker

Kali Linux runs on 192.168.100.40 with a static address and deliberately no default gateway, so it cannot reach the Internet and cannot be used outside the lab. It is the source of the external attack simulations.

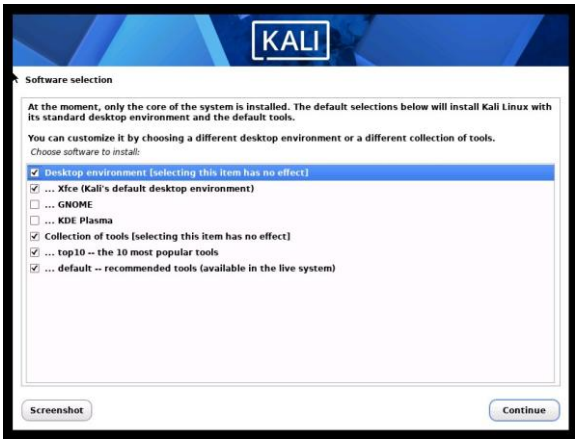


Figure 58: Kali Linux software selection

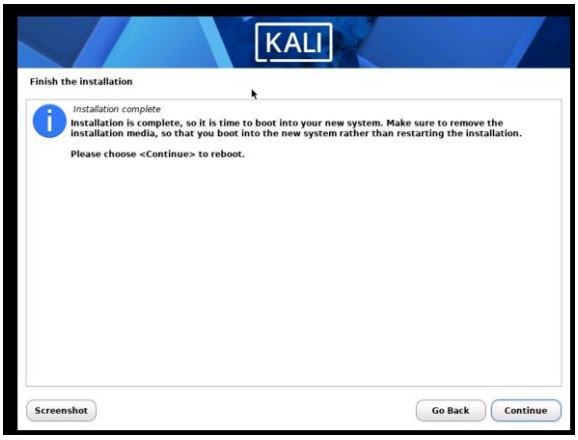


Figure 59: Kali Linux installation complete

4.8 Resource footprint

With every virtual machine running, the 32 GB laptop used roughly 22 GB of memory, which left sufficient headroom for the Python monitor and the model.

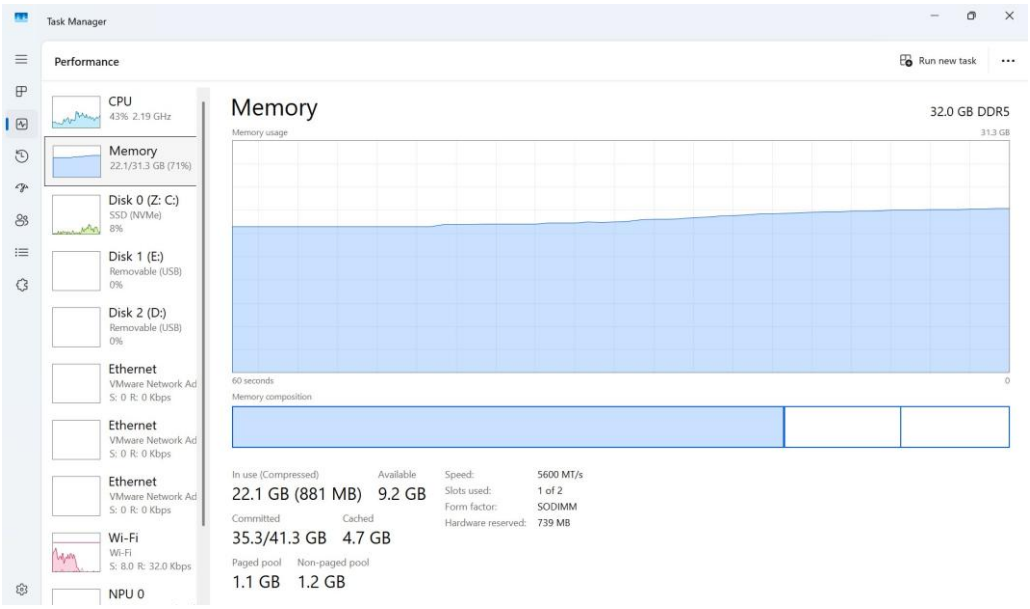


Figure 60: Laptop memory use with all lab VMs running

5. Commands and Verified Outputs

This chapter lists the commands that were run successfully on each machine of the lab: the laptop, the Windows 11 virtual machine, the Windows Server 2019 domain controller, the RHEL server and the Kali attacker. For every command it explains what it does and shows the output exactly as the terminal printed it. Passwords are never displayed: they are typed at hidden prompts.

Prompt conventions: (venv) PS C:\catavo-ml> is Windows PowerShell on the laptop with the Python virtual environment active; PS C:\WINDOWS\system32> is an Administrator PowerShell inside a Windows virtual machine; yashsahu@catavo-keycloak:~$ is the RHEL terminal.

5.1 Laptop: project environment (Windows PowerShell)

All machine-learning and SOAR code lives in the folder C:\catavo-ml on the laptop. The commands below open the folder, activate an isolated Python environment and install the libraries the scripts need.

Table 3: Environment commands

Command	What it does
cd C:\catavo-ml	Moves the shell into the project folder that holds the scripts and the data subfolder.
.\venv\Scripts\Activate.ps1	Activates the Python virtual environment (venv) so that every package is installed into the project and not into the whole system. The prompt gains the prefix (venv).
python monitor.py	Starts the real-time monitor (explained in Section 5.7).
pip install pandas numpy scikit-learn requests paramiko matplotlib	Installs the libraries used by the project: pandas and numpy (tables and arrays), scikit-learn (Isolation Forest baseline), requests (Splunk and Keycloak REST calls), paramiko (SSH to pfSense and RHEL for the response), matplotlib (training-loss chart). PyTorch for the autoencoder was installed in the same environment.

Output of the library installation (unchanged)

```
(venv) PS C:\catavo-ml> pip install pandas numpy scikit-learn requests paramiko matplotlib
Collecting pandas
  Downloading pandas-3.0.6-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting numpy
  Downloading numpy-2.5.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.1-cp313-cp313-win_amd64.whl.metadata (9.3 kB)
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting paramiko
  Downloading paramiko-5.0.0-py3-none-any.whl.metadata (3.7 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.2-cp313-cp313-win_amd64.whl.metadata (80 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.5-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting scipy>=1.10.0 (from scikit-learn)
  Downloading scipy-1.18.1-cp313-cp313-win_amd64.whl.metadata (61 kB)
Collecting joblib>=1.4.0 (from scikit-learn)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting narwhals>=2.0.1 (from scikit-learn)
  Downloading narwhals-2.27.1-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn)
  Downloading threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.5.2-cp313-cp313-win_amd64.whl.metadata (47 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Collecting bcrypt>=3.2 (from paramiko)
  Downloading bcrypt-5.0.0-cp39-abi3-win_amd64.whl.metadata (10 kB)
Collecting cryptography>=3.3 (from paramiko)
  Downloading cryptography-50.0.2-cp311-abi3-win_amd64.whl.metadata (4.4 kB)
Collecting invoke>=2.0 (from paramiko)
  Downloading invoke-3.0.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pynacl>=1.5 (from paramiko)
  Downloading pynacl-1.6.2-cp38-abi3-win_amd64.whl.metadata (10 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.4.0-cp313-cp313-win_amd64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
```

```
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.66.1-cp313-cp313-win_amd64.whl.metadata (132 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp313-cp313-win_amd64.whl.metadata (5.2 kB)
Collecting packaging>=20.0 (from matplotlib)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp313-cp313-win amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Collecting cffi>=2.0.0 (from cryptography>=3.3->paramiko)
  Downloading cffi-2.1.1-cp313-cp313-win amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography>=3.3->paramiko)
  Downloading pycparser-3.11-py3-none-any.whl.metadata (8.2 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.4.0->scikit-learn)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading pandas-3.0.6-cp313-cp313-win amd64.whl (9.6 MB)
   ---------------------------------------- 9.6/9.6 MB 1.2 MB/s  0:00:07
Downloading numpy-2.5.4-cp313-cp313-win_amd64.whl (12.6 MB)
   ---------------------------------------- 12.6/12.6 MB 1.2 MB/s  0:00:10
Downloading scikit_learn-1.9.1-cp313-cp313-win_amd64.whl (8.2 MB)
   ---------------------------------------- 8.2/8.2 MB 1.3 MB/s  0:00:06
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.2-cp313-cp313-win amd64.whl (206 kB)
Downloading idna-3.20-py3-none-any.whl (69 kB)
Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
Downloading paramiko-5.0.0-py3-none-any.whl (208 kB)
Downloading matplotlib-3.11.2-cp313-cp313-win amd64.whl (9.3 MB)
   ---------------------------------------- 9.3/9.3 MB 1.5 MB/s  0:00:06
Downloading bcrypt-5.0.0-cp39-abi3-win amd64.whl (150 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Downloading contourpy-1.4.0-cp313-cp313-win_amd64.whl (234 kB)
Downloading cryptography-50.0.2-cp311-abi3-win_amd64.whl (3.8 MB)
   ---------------------------------------- 3.8/3.8 MB 1.4 MB/s  0:00:02
Downloading cffi-2.1.1-cp313-cp313-win amd64.whl (185 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.66.1-cp313-cp313-win_amd64.whl (2.5 MB)
   ---------------------------------------- 2.5/2.5 MB 867.7 kB/s  0:00:02
Downloading invoke-3.0.3-py3-none-any.whl (160 kB)
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading kiwisolver-1.5.1-cp313-cp313-win_amd64.whl (70 kB)
Downloading narwhals-2.27.1-py3-none-any.whl (483 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Downloading pillow-12.3.0-cp313-cp313-win_amd64.whl (7.2 MB)
   ---------------------------------------- 7.2/7.2 MB 1.1 MB/s  0:00:06
Downloading pynacl-1.6.2-cp38-abi3-win amd64.whl (239 kB)
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading scipy-1.18.1-cp313-cp313-win amd64.whl (36.6 MB)
   ---------------------------------------- 36.6/36.6 MB 1.1 MB/s  0:00:33
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)
Downloading pycparser-3.11-py3-none-any.whl (51 kB)
Downloading tzdata-2026.5-py2.py3-none-any.whl (347 kB)
Installing collected packages: urllib3, tzdata, threadpoolctl, six, pyparsing, pycparser, pillow, packaging, numpy,
narwhals, kiwisolver, invoke, idna, fonttools, cycler, cloudpickle, charset_normalizer, certifi, bcrypt, scipy, requests,
python-dateutil, joblib, contourpy, cffi, scikit-learn, pynacl, pandas, matplotlib, cryptography, paramiko
Successfully installed bcrypt-5.0.0 certifi-2026.7.22 cffi-2.1.1 charset_normalizer-3.5.2 cloudpickle-3.1.2 contourpy-1.4.0
cryptography-50.0.2 cycler-0.12.1 fonttools-4.66.1 idna-3.20 invoke-3.0.3 joblib-1.6.0 kiwisolver-1.5.1 matplotlib-3.11.2
narwhals-2.27.1 numpy-2.5.4 packaging-26.3 pandas-3.0.6 paramiko-5.0.0 pillow-12.3.0 pycparser-3.11 pynacl-1.6.2 pyparsing-
3.3.3 python-dateutil-2.9.0.post0 requests-2.34.2 scikit-learn-1.9.1 scipy-1.18.1 six-1.17.0 threadpoolctl-3.7.0 tzdata-
2026.5 urllib3-2.8.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\catavo-ml>
```

The line "Successfully installed ..." confirms that every package and its dependencies were installed. The final "notice" lines only announce that a newer version of pip exists and can be ignored.

Starting the monitor (dry run)

```
PS C:\WINDOWS\system32> cd C:\catavo-ml
PS C:\catavo-ml> .\venv\Scripts\Activate.ps1
(venv) PS C:\catavo-ml> python monitor.py
Splunk username [admin]: 25168014
Splunk password for 25168014:
monitor started (dry run), threshold 0.07598. Ctrl+C to stop.
```

The monitor asks for the Splunk user name and the password. The password is typed at a hidden prompt and is not echoed. The line "monitor started (dry run)" shows that the model, the threshold and the connection to Splunk loaded correctly; in dry-run mode it scores every login but only prints "would_respond" instead of acting. This first run used the earlier five-feature model, whose threshold was 0.07598; the final model threshold is 0.02303.

5.2 Laptop: building the dataset and training the models

After the normal-behaviour data had been exported from Splunk, the final scripts were copied into the project folder and run in sequence. The commands and their meaning are listed first and the output of each step follows.

Table 4: Dataset and training commands

Command	What it does
move -Force $HOME\Downloads\features2.py C:\catavo-ml\	Moves the downloaded script into the project folder; -Force overwrites an older copy. The same pattern was used for catavo_ml.py, baseline_if.py and evaluate.py.
python features2.py	Cleans the Splunk export, selects the two normal batches, separates the 80 hold-out logins, builds the four model features and writes features.csv, baseline_ips.json and keycloak_normal_split.csv. Old files are first backed up in data\v1.
python catavo_ml.py --epochs 1000	Trains the autoencoder for 1000 epochs, sets the alert threshold and saves model.pt, scaler.json, threshold.json and the loss chart.
python baseline_if.py	Trains the Isolation Forest baseline on the same data and split.
copy ...\eval_output.txt ...\eval_output_5feature.txt	Keeps the earlier five-feature evaluation as a separate file so that it can be compared with the final one.
python evaluate.py | Tee-Object C:\catavo-ml\data\eval_output.txt	Scores all 54 attack runs and the normal data with every detector. Tee-Object shows the result on screen and saves the same text to eval_output.txt.

python features2.py

```
(venv) PS C:\catavo-ml> python features2.py
old files copied to C:\catavo-ml\data\v1
events kept per window and type:
B1 {('LOGIN', 'train'): 600, ('LOGIN_ERROR', 'train'): 38, ('LOGOUT', 'train'): 600}
B2 {('LOGIN', 'holdout'): 80, ('LOGIN', 'train'): 120, ('LOGIN_ERROR', 'holdout'): 2, ('LOGIN_ERROR', 'train'): 3,
('LOGOUT', 'holdout'): 80, ('LOGOUT', 'train'): 120}


training rows: 720  per batch: {'B1': 600, 'B2': 120}
                      mean    std   min     max
login_hour           17.17   3.49  9.33   20.82
session_duration     33.44  15.31  3.06   59.09
failed_before_login   0.38   0.59  0.00    3.00
new_ip                0.00   0.00  0.00    0.00
login_gap            50.96  41.81  5.15  998.64
holdout logins: 80
```

Reading the output: batch B1 (evening) contributed 600 normal logins and batch B2 (daytime) 120 training logins plus 80 hold-out logins that are never used for training. In total 720 training rows were produced. The statistics table shows that new_ip is always 0 in normal data (every normal login comes from the user's own machine), session duration averages about 33 seconds and the gap between logins about 51 seconds.

python catavo_ml.py --epochs 1000

```
(venv) PS C:\catavo-ml> python catavo_ml.py --epochs 1000
epoch   1  train 0.20995  val 0.19241
epoch 100  train 0.02136  val 0.02261
epoch 200  train 0.00370  val 0.00414
epoch 300  train 0.00228  val 0.00225
epoch 400  train 0.00218  val 0.00206
epoch 500  train 0.00213  val 0.00195
epoch 600  train 0.00209  val 0.00188
epoch 700  train 0.00206  val 0.00183
epoch 800  train 0.00204  val 0.00179
epoch 900  train 0.00203  val 0.00176
epoch 1000  train 0.00202  val 0.00174
threshold (99th pct of training error) = 0.02303
validation false-positive rate = 0.7%  (1 of 144)
error mean 0.00202  max 0.09153
saved model.pt, scaler.json, threshold.json, training_loss.png in C:\catavo-ml\data
```

The training error falls from 0.21 at epoch 1 to 0.002 at epoch 1000 and the validation error follows it closely (0.00174), so the model is not over-fitting. The threshold is the 99th percentile of the errors on normal training data, 0.02303: by construction only about one normal login in a hundred can exceed it. On the validation split 1 login in 144 (0.7 %) did.

python baseline_if.py

```
(venv) PS C:\catavo-ml> python baseline_if.py
threshold (99th pct of training score) = 0.6628
validation false-positive rate = 1.4%  (2 of 144)
saved if_model.joblib and if_threshold.json in C:\catavo-ml\data
```

The Isolation Forest uses the same rule to set its threshold (0.6628) and flags 2 of 144 validation logins (1.4 %).

python evaluate.py | Tee-Object ... (first part of the output)

```
(venv) PS C:\catavo-ml> python evaluate.py | Tee-Object C:\catavo-ml\data\eval_output.txt
WARNING: 1 runs have no scored events (excluded): ['brute_force-catavo-kali-1014015052-1']...


attack attempts scored: 420 in 54 runs;  normal attempts: 761 in-sample, 82 holdout
in-sample FPR by batch (evening B1 / daytime B2):
    Autoencoder  Isolation Forest
B1          0.2               0.2
B2          4.9               0.8


score check  AE threshold 0.0230: normal median 0.0007 max 0.0908; attack median 0.3117
             IF threshold 0.6628: normal median 0.4337 max 0.6829; attack median 0.6530 max 0.6536
```

The evaluation scored 420 attack attempts from 54 attack runs and 843 normal attempts (761 used in training and 82 hold-out). The line starting with WARNING only says that one run listed in the run file left no events in Splunk and was therefore left out; the 54 scored runs are the ones reported. The score check shows the separation that makes the detector work: normal logins have a median error of 0.0007 and a maximum of 0.0908, while attacks have a median of 0.3117, far above the threshold 0.0230.

python evaluate.py (summary table)

```
=== Overall (rates in %) ===
                          in-sample FPR %  holdout FPR %  attempt recall %  runs detected %
Autoencoder                           0.9            2.4              82.6            83.3
AE, gap frozen                        0.0            0.0              69.8            72.2
AE, new_ip frozen                     0.9            2.4              57.1            48.1
AE, failed-logins frozen              0.9            1.2              67.1            72.2
Isolation Forest                      0.3            2.4               0.0             0.0
Isolation Forest, 95th pct            4.9            8.5              64.3            59.3
Rule 1 (brute force)                  0.0            0.0              37.6            48.1
Rule 2 (unknown IP)                   0.0            0.0              15.7            22.2
Rules 1 OR 2                          0.0            0.0              53.3            70.4
```

The summary table is the main result: the autoencoder detects 83.3 % of the attack runs with 0.9 % false positives on training-period data and 2.4 % on unseen hold-out data. The remaining rows are the ablations (one feature frozen at a time), the Isolation Forest, and the two Splunk rules. The complete output, including the table of detections per scenario, is reproduced in Appendix B.

5.3 Windows 11 virtual machine (CATAVO-WS01): normal-behaviour generator

To teach the model what normal looks like, user1 was simulated on the Windows 11 machine with a PowerShell loop that signs in to Keycloak through the catavo-sim client, stays logged in for a while and signs out, 100 times with human-like random pauses. The commands were typed in an Administrator PowerShell window.

Setting up the variables

```
PS C:\WINDOWS\system32> $u="user1"
PS C:\WINDOWS\system32> $pw=[Net.NetworkCredential]::new("",(Read-Host "Password" -AsSecureString)).Password
Password: **********
PS C:\WINDOWS\system32> $n=100
PS C:\WINDOWS\system32> $kc="http://192.168.100.20:8080/realms/catavo/protocol/openid-connect"
```

Table 5: Generator variables

Command	What it does
$u="user1"	Stores the user name that will sign in.
$pw=[Net.NetworkCredential]::new("",(Read-Host "Password" -AsSecureString)).Password	Asks for the password at a hidden prompt (shown as **********) and keeps it only in memory, so it never appears on screen or in the command history.
$n=100	Number of sessions to simulate.
$kc="http://192.168.100.20:8080/realms/catavo/protocol/openid-connect"	Base address of the Keycloak OpenID Connect endpoints for realm catavo.

The generator loop (one session per iteration)

```
PS C:\WINDOWS\system32> for($i=1;$i -le $n;$i++){
>>   $r=Get-Random -Maximum 100
>>   if($r -lt 25){$p=Get-Random -Minimum 1 -Maximum 5}elseif($r -lt 90){$p=Get-Random -Minimum 5 -Maximum
25}else{$p=Get-Random -Minimum 60 -Maximum 180}
>>   Start-Sleep -Seconds $p
>>   if((Get-Random -Maximum 15) -eq 0){
>>     try{Invoke-RestMethod -Method Post -Uri "$kc/token" -Body @{client_id="catavo-
sim";grant_type="password";username=$u;password="wrong$i";scope="openid"}}catch{}
>>     Start-Sleep -Seconds (Get-Random -Minimum 3 -Maximum 10)
>>   }
>>   $t=Invoke-RestMethod -Method Post -Uri "$kc/token" -Body @{client_id="catavo-
sim";grant_type="password";username=$u;password=$pw;scope="openid"}
>>   if((Get-Random -Maximum 100) -lt 25){$d=Get-Random -Minimum 3 -Maximum 10}else{$d=Get-Random -Minimum 10 -
Maximum 60}
>>   Start-Sleep -Seconds $d
>>   Invoke-RestMethod -Method Post -Uri "$kc/logout" -Body @{client_id="catavo-
sim";refresh_token=$t.refresh_token} | Out-Null
>>   Write-Host "session $i done at $(Get-Date -Format HH:mm:ss)"
>> }
```

Table 6: Explanation of the generator loop

Line of the loop	What it does
for($i=1;$i -le $n;$i++){	Repeats the block once per session, from 1 to 100.
$r=Get-Random -Maximum 100 ... Start-Sleep -Seconds $p	Chooses a random waiting time before the next session: 25 % of the time 1 to 4 seconds, 65 % of the time 5 to 24 seconds and 10 % of the time 60 to 179 seconds. This gives the irregular rhythm of a real user.
if((Get-Random -Maximum 15) -eq 0){ ... password="wrong$i" ... }	About one session in fifteen starts with a mistyped password (a LOGIN_ERROR event), followed by a pause of 3 to 9 seconds, as a human would retype it.
$t=Invoke-RestMethod -Method Post -Uri "$kc/token" -Body @{client_id="catavo-sim";grant_type="password";...}	The actual login: requests a token from Keycloak with the correct password. Keycloak records a LOGIN event with the user, session and source IP address.
$d=... Start-Sleep -Seconds $d	Keeps the session open: 25 % of the time 3 to 9 seconds, otherwise 10 to 59 seconds. This produces the session-duration feature.
Invoke-RestMethod -Method Post -Uri "$kc/logout" -Body @{...refresh_token=$t.refresh_token} | Out-Null	Signs out with the refresh token. Keycloak records a LOGOUT event, which lets the pipeline measure the session length.
Write-Host "session $i done at $(Get-Date -Format HH:mm:ss)"	Prints a progress line after each session.

```
session 1 done at 13:20:05                    session 51 done at 14:06:17
session 2 done at 13:21:05                    session 52 done at 14:06:51
session 3 done at 13:21:53                    session 53 done at 14:07:03
session 4 done at 13:22:57                    session 54 done at 14:07:25
session 5 done at 13:23:27                    session 55 done at 14:07:54
session 6 done at 13:24:41                    session 56 done at 14:08:26
session 7 done at 13:25:10                    session 57 done at 14:08:51
session 8 done at 13:26:04                    session 58 done at 14:09:16
```

```
session 9 done at 13:27:56
session 10 done at 13:30:53
session 11 done at 13:31:58
session 12 done at 13:32:23
session 13 done at 13:33:08
session 14 done at 13:35:24
session 15 done at 13:35:40
session 16 done at 13:36:35
session 17 done at 13:37:11
session 18 done at 13:37:48
session 19 done at 13:38:26
session 20 done at 13:39:13
session 21 done at 13:39:24
session 22 done at 13:39:52
session 23 done at 13:40:23
session 24 done at 13:41:25
session 25 done at 13:43:33
session 26 done at 13:43:53
session 27 done at 13:44:18
session 28 done at 13:44:36
session 29 done at 13:45:05
session 30 done at 13:45:34
session 31 done at 13:45:59
session 32 done at 13:46:47
session 33 done at 13:48:30
session 34 done at 13:49:17
session 35 done at 13:50:00
session 36 done at 13:50:39
session 37 done at 13:50:58
session 38 done at 13:51:37
session 39 done at 13:54:45
session 40 done at 13:55:24
session 41 done at 13:56:30
session 42 done at 13:59:36
session 43 done at 14:00:37
session 44 done at 14:01:19
session 45 done at 14:02:12
session 46 done at 14:03:13
session 47 done at 14:04:17
session 48 done at 14:04:30
session 49 done at 14:04:49
session 50 done at 14:05:16
session 59 done at 14:10:23
session 60 done at 14:10:34
session 61 done at 14:10:55
session 62 done at 14:11:08
session 63 done at 14:11:33
session 64 done at 14:12:34
session 65 done at 14:13:25
session 66 done at 14:14:25
session 67 done at 14:15:27
session 68 done at 14:15:44
session 69 done at 14:16:29
session 70 done at 14:17:30
session 71 done at 14:17:48
session 72 done at 14:18:22
session 73 done at 14:19:29
session 74 done at 14:20:30
session 75 done at 14:21:28
session 76 done at 14:22:12
session 77 done at 14:23:00
session 78 done at 14:23:22
session 79 done at 14:24:04
session 80 done at 14:24:32
session 81 done at 14:24:43
session 82 done at 14:24:54
session 83 done at 14:25:22
session 84 done at 14:26:21
session 85 done at 14:26:57
session 86 done at 14:27:20
session 87 done at 14:28:04
session 88 done at 14:28:43
session 89 done at 14:29:33
session 90 done at 14:29:46
session 91 done at 14:30:31
session 92 done at 14:31:32
session 93 done at 14:32:26
session 94 done at 14:33:11
session 95 done at 14:33:36
session 96 done at 14:33:54
session 97 done at 14:35:03
session 98 done at 14:35:24
session 99 done at 14:35:52
session 100 done at 14:37:01
```

The output proves that all 100 sessions completed without a failed request: session 1 finished at 13:20:05 and session 100 at 14:37:01, an average of about 49 seconds per session. The same script (with user2 and the laptop as the source) produced the second user's data.

Checking the Splunk Universal Forwarder service

```
PS C:\WINDOWS\system32> Get-Service Splunk*

Status   Name               DisplayName
------   ----               -----------
Running  SplunkForwarder    SplunkForwarder
```

Get-Service lists Windows services whose name starts with "Splunk". The status Running shows that the Universal Forwarder is active on the workstation and sending the Windows logs to Splunk.

5.4 Windows Server 2019 (CATAVO-AD): domain controller

The domain controller was administered from an Administrator PowerShell window. The following command confirmed the identity of the server after installation and promotion.

Checking the computer name

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> hostname
CATAVO-AD
PS C:\Users\Administrator>
```

Table 7: Domain controller command

Command	What it does
hostname	Prints the computer name of the machine. The answer CATAVO-AD confirms that this is the domain controller of the domain catavo.local. The same window shows the operating system banner "Windows Server 2019 Standard Evaluation".

The domain controller's Application, Security and System logs, including logon events (4624), are forwarded to the ad index of Splunk by the Universal Forwarder, as shown in Chapter 4 and Chapter 6.

5.5 RHEL 10.2 server (catavo-keycloak): network and Splunk start

On the RHEL machine the network configuration was verified, and Splunk Enterprise was started under its own service account. The outputs below are copied from the terminal screenshots.

Table 8: RHEL commands

Command	What it does
ip a	Lists the network interfaces and their addresses. Interface ens160 holds 192.168.100.20/24 and the loopback interface holds 127.0.0.1.
ip route	Prints the routing table: the default route goes through the pfSense LAN address 192.168.100.1.
ping -c 3 192.168.100.1	Sends three test packets to the firewall to confirm that the server can reach the gateway.
ping -c 3 8.8.8.8	Isolation check: three packets to a public address. The 100 % packet loss confirms that the lab has no Internet access, as designed.
sudo -u splunk /opt/splunk/bin/splunk start --accept-license	Starts Splunk Enterprise as the user splunk and accepts the license. Splunk checks its configuration and files, creates a certificate and then waits for its web interface on port 8000.

ip a and ip route

```
yashsahu@catavo-keycloak:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:7f:a1:32 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altname enx000c297fa132
    inet 192.168.100.20/24 brd 192.168.100.255 scope global noprefixroute ens160
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe7f:a132/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
yashsahu@catavo-keycloak:~$ ip route
default via 192.168.100.1 dev ens160 proto static metric 100
192.168.100.0/24 dev ens160 proto kernel scope link src 192.168.100.20 metric 100
```

Gateway reachable, Internet unreachable (isolated lab)

```
yashsahu@catavo-keycloak:~$ ping -c 3 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=0.493 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=1.24 ms

--- 192.168.100.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2067ms
rtt min/avg/max/mdev = 0.493/0.944/1.237/0.323 ms
yashsahu@catavo-keycloak:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

```
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2065ms
```

Three of three packets reached the gateway in about one millisecond, so the server is correctly attached to the lab network, while nothing leaves the lab. The server address 192.168.100.20 and the default gateway 192.168.100.1 match the design in Chapter 3.

Starting Splunk: sudo -u splunk /opt/splunk/bin/splunk start --accept-license (key lines)

```
Checking conf files for problems...
        Done
        Checking default conf files for edits...
        Validating installed files against hashes from '/opt/splunk/splunk-10.6.0.5-86587d4e3b27-linux-amd64-
manifest'
        All installed files intact.
        Done
All preliminary checks passed.

Starting splunk server daemon (splunkd)...
Using configuration from /opt/splunk/share/openssl3/openssl.cnf
Certificate request self-signature ok
subject=CN=catavo-keycloak, O=SplunkUser
Done

Waiting for web server at http://127.0.0.1:8000 to be available...
```

"All preliminary checks passed" and "All installed files intact" show a healthy installation of Splunk Enterprise 10.6.0.5; after the web server came up the Splunk home page opened (Chapter 4).

5.6 Kali Linux: attack commands

The attacks are launched with one Python script, attacks.py, which uses only the standard library and therefore runs on Kali without any installation. Its command line, taken from the script, is:

Command syntax

```
python3 attacks.py <scenario> [--user user1] [--runs 1] [--pause 120]

scenarios: brute_force, spray, slow_low, stolen_credential, token_burst
```

Table 9: attacks.py arguments

Argument	Meaning
<scenario>	Which of the five attacks to run (described in Chapter 10).
--user user1	Account that is attacked (default user1).
--runs N	How many repeated runs to execute in one call.
--pause 120	Seconds to wait between two runs, so that runs do not overlap.

For the two scenarios that use a valid password (stolen_credential and token_burst) the script asks for it at a hidden prompt. Every run is recorded in attack_runs.csv with a run identifier, the scenario, the host name, the user and the start and end time; this file is the ground truth used by the evaluation. The effect of a run is visible in Splunk, for example the 17 failed logins of a brute-force run from the Kali address shown in Chapter 10.

5.7 Laptop: response engine and monitor

Table 10: Response commands

Command	What it does
python monitor.py	Dry run: scores every login and prints the decision but changes nothing.

Command	What it does
python monitor.py --act	Real response: for each flagged login revokes the Keycloak session and blocks the source address on the RHEL firewall and on pfSense.
python monitor.py --act --auto-unblock 20	Test mode used for the latency measurement: the block is removed 20 seconds after it was applied, so that the next trial can start from a clean state.
python soar.py status	Checks the three connections: the Keycloak admin token, the pfSense blocked_ips table and the RHEL rich rules.
python soar.py block <ip> <userId>	Applies the response manually to one address and user.
python soar.py unblock <ip>	Removes the RHEL and pfSense blocks for an address.

After the test runs the status command confirmed a clean state: the Keycloak administrator token was obtained, the pfSense blocked_ips table was empty and the RHEL firewall had no rich rules, which proves that every automatic block had been released again.

Table 11: Result of python soar.py status

Item	Result recorded in the run
keycloak admin token	OK
pfSense blocked_ips	(empty)
RHEL rich rules	(none)

The latency test used ten single wrong-password attempts from Kali, 45 seconds apart, with the monitor running in act mode. The statistics of the ten measurements of event_to_done_s (seconds from the Keycloak event to the end of the response) were:

Table 12: Latency statistics (seconds)

count	mean	std	min	25 %	50 % (median)	75 %	max
10	7.174	2.657	4.36	5.3125	6.505	8.9125	11.86

The export used to build the datasets was checked before use: the Splunk CSV export had 2351 rows and 6 columns, made up of 993 LOGIN, 992 LOGOUT and 366 LOGIN_ERROR events.

6. Telemetry Pipeline

Every component writes to Splunk, which acts as the single source of truth for the detection code. Table 3 summarises the sources.

Table 13: Telemetry sources

Source	Transport	Splunk index	Content
pfSense	Syslog UDP 5514	pfsense	Firewall and system events
Domain controller	Universal Forwarder, TCP 9997	ad	Windows Application, Security and System logs, including logon events
Windows 11 endpoint	Universal Forwarder, TCP 9997	endpoint	Windows Application, Security and System logs
Keycloak	Server log read by Splunk	keycloak	LOGIN, LOGIN_ERROR, LOGOUT, CODE_TO_TOKEN events with user, session, IP and error

Keycloak events are the primary input of the model. A search by event type shows the four event types received, and a table view shows the fields later used for feature extraction.

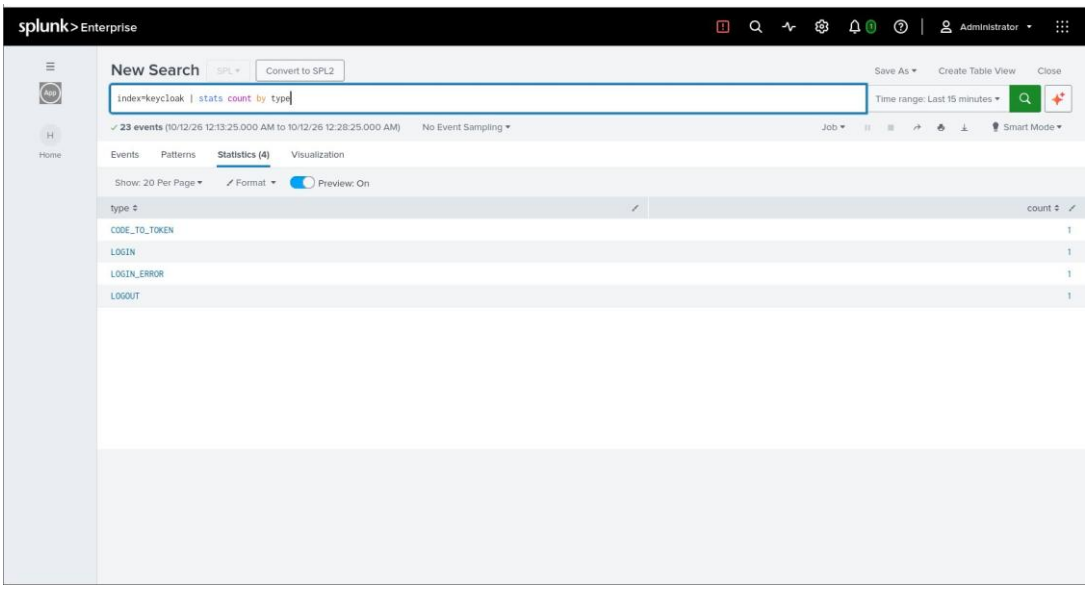


Figure 61: Keycloak event types received in Splunk

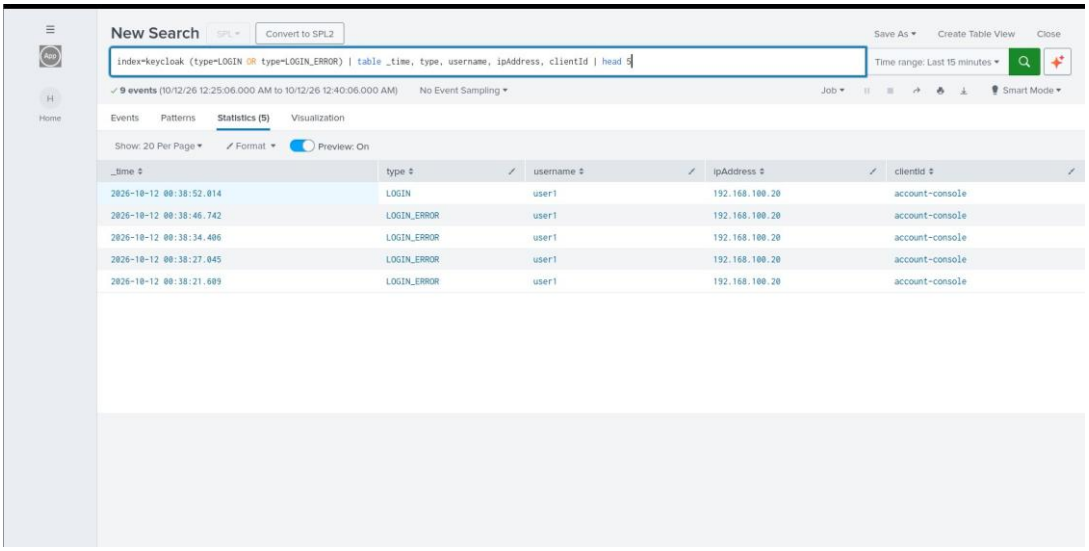


Figure 62: Keycloak LOGIN and LOGIN_ERROR events in Splunk

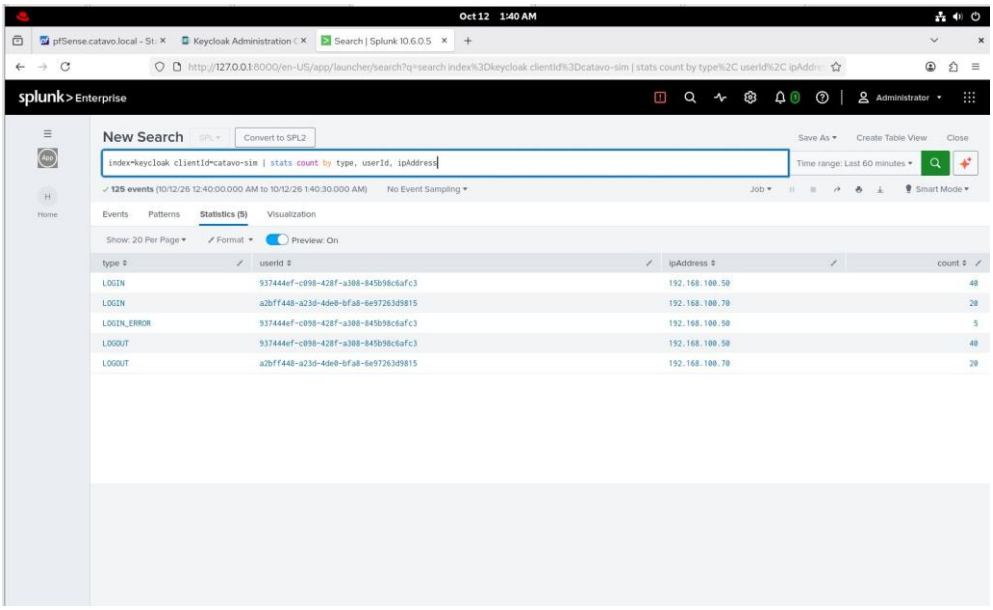


Figure 63: Keycloak events per user and source IP during baseline collection

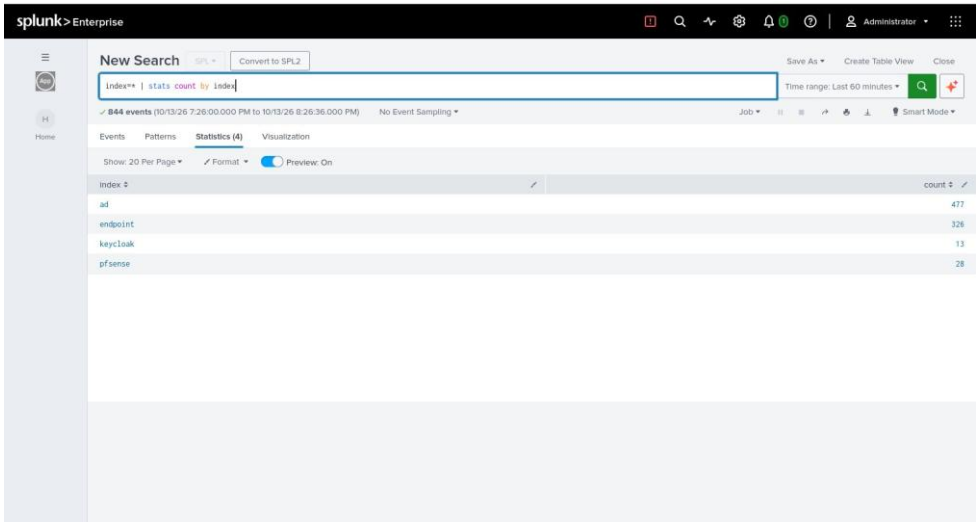


Figure 64: Event counts per Splunk index

The monitor retrieves events through the Splunk REST interface (port 8089) every three seconds. Searching is done with SPL on index=keycloak; timestamps are parsed as ISO 8601 and only events newer than the monitor start are processed.

7. Data Collection and Feature Engineering

7.1 Normal-behaviour data

A data generator on the endpoint logs in as user1 (from 192.168.100.50) and as user2 (from the laptop, 192.168.100.70) with realistic pauses between sessions, session durations and occasional mistyped passwords. Each session produces a login, a short period of activity and a logout.

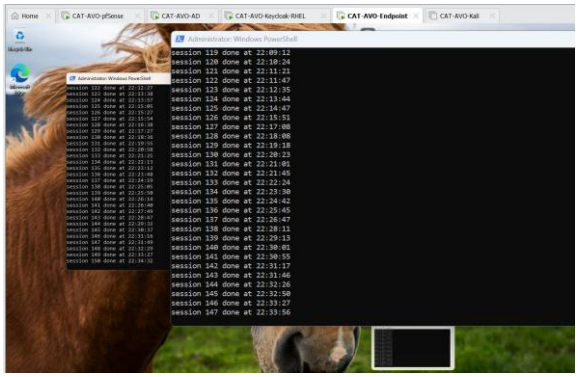


Figure 65: Data generator running on the workstation

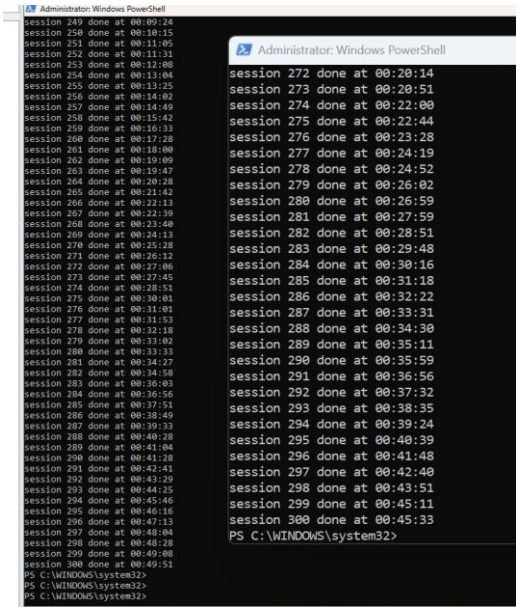


Figure 66: Data generator finishing session 300

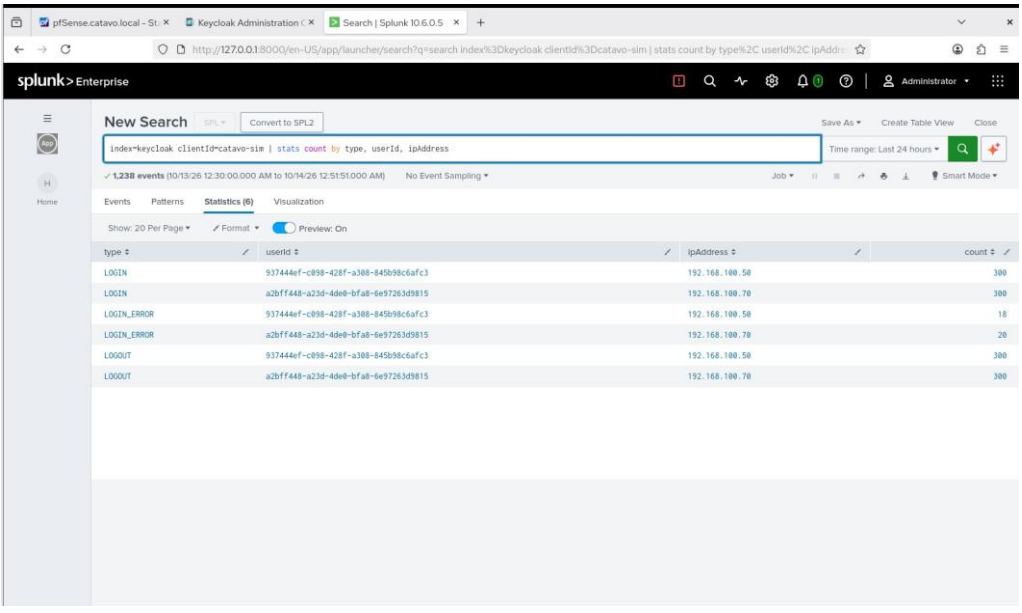


Figure 67: Collected Keycloak events per user

Normal data were collected in two batches. Batch B1 covers the evening of 13 October 2026 (20:00 to 01:00 IST). Batch B2 was collected on the afternoon of 14 October (13:19 to 15:00) so that the model also sees daytime behaviour. The Keycloak events were exported from Splunk as CSV.

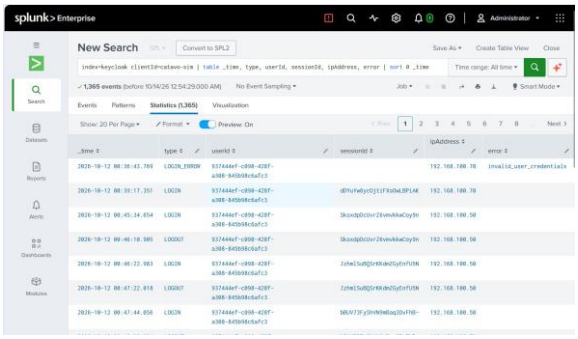


Figure 68: Event table prepared for export

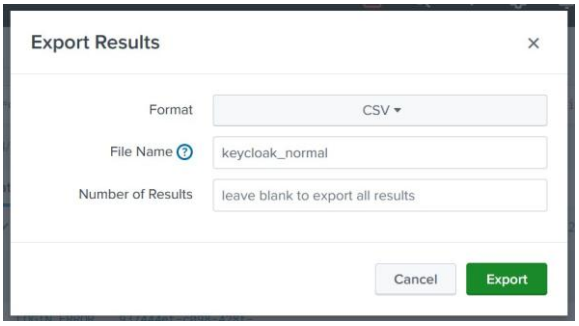


Figure 69: Exporting the normal-behaviour data as CSV

7.2 Features

Each login is turned into four numerical features. The hour of the day is still recorded in the logs but is not used by the model: with narrow, regular normal windows it made the model flag ordinary daytime logins as anomalies (see Chapter 14).

Table 14: Model features

Feature	Definition	Why it matters
session_duration	Seconds between a login and its logout	Hijacked or scripted sessions are very short or very long
failed_before_login	Failed attempts by the user before this login	Guessing and spraying leave failed attempts
new_ip	1 if the source address is not in the user's baseline addresses, otherwise 0	Credential use from a new location
login_gap	Seconds since the user's previous login, transformed with log(1+x)	Bursts of logins are unusual for a human

7.3 Dataset split

The training set contains 720 normal logins (600 from B1 and 120 from B2). The last 40 logins of each user in B2 (80 logins, 82 attempts) were held out and never used for training. They provide an honest estimate of the false-positive rate on unseen normal behaviour.

8. Machine-Learning Model

8.1 Autoencoder

An autoencoder learns to compress its input into a small code and rebuild it. Trained only on normal logins, it reconstructs normal behaviour well and abnormal behaviour poorly, so the reconstruction error acts as an anomaly score. The diagram below shows the principle as originally designed with five features; the final model uses the four features of Chapter 7.

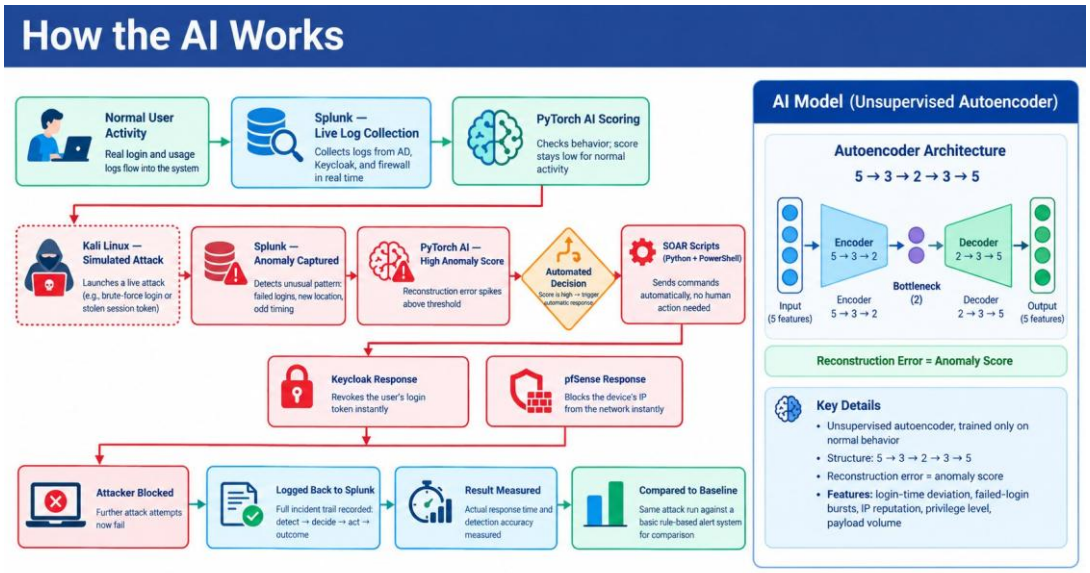


Figure 70: How the autoencoder works (original five-feature design; the final model uses four features)

Table 15: Autoencoder configuration

Parameter	Value
Architecture	4 - 3 - 2 - 3 - 4 (fully connected, Tanh activations)
Loss and optimiser	Reconstruction error, Adam with learning rate 0.01
Training	1000 epochs on 720 normal logins
Scaling	Min-max, fitted on the training data
Split	80/20 random split of the training data, seed 42
Threshold	99th percentile of the training reconstruction error = 0.02303
Decision	A login whose error exceeds the threshold is an anomaly

8.2 Baselines

- **Splunk rules.** Rule 1: five or more LOGIN_ERROR events from one IP within five minutes. Rule 2: a LOGIN from an address outside the two known addresses (192.168.100.50 and 192.168.100.70).
- **Isolation Forest.** 200 trees on the same four features, scaling and split; threshold at the 99th percentile of training scores (0.6628). A looser 95th-percentile threshold is also reported.

8.3 Program files

Each file is explained in detail in Chapter 11 and listed in full in Appendix A.

Table 16: Project code

File	Purpose
features2.py	Builds the feature table from the Keycloak export for batches B1 and B2 and the hold-out split
catavo_ml.py	Trains the autoencoder and saves the model, scaler, threshold and loss curve
baseline_if.py	Trains the Isolation Forest baseline
soar.py	Containment actions: Keycloak session revoke, RHEL firewall block, pfSense block, status and unblock
monitor.py	Polls Splunk every 3 s, scores new logins and calls the SOAR module (dry-run or --act)
attacks.py	Five attack scenarios used for evaluation
evaluate.py	Runs all detectors on the attack runs and the normal data and writes the result tables

9. Automated Response (SOAR)

When the monitor flags a login, the response engine takes three coordinated actions against the offending user and source address.

- **Keycloak:** the user's sessions are revoked through the Admin REST API (users/{id}/logout), which ends the session at the identity provider.
- **RHEL host firewall:** a firewalld rich rule drops traffic from the source address. The engine uses a restricted account (soar) with a narrow sudoers rule.
- **pfSense:** the address is added to the blocked_ips table and existing states are killed (pfctl -t blocked_ips -T add <ip>; pfctl -k <ip>).

Addresses of infrastructure (192.168.100.1, .10, .20, .30 and .70) are protected: for them only the session revocation is performed, so the engine can never lock out the firewall, the directory, the identity provider, Splunk or the operator's own laptop. The monitor runs in dry-run mode by default; the --act switch enables real actions and --auto-unblock N releases blocks after N seconds.

Design finding. pfSense can only filter traffic that crosses it. Traffic between two machines on the same segment does not, so a pfSense block alone does not stop an attacker on the same network from reaching Keycloak. The host firewall block on the RHEL machine was added for this reason, and the pfSense block remains useful for traffic that does cross the firewall.

10. Attack Simulation

Attacks were launched against realm catavo with the client catavo-sim from two sources: the Kali machine (an unknown address) and the laptop acting as user2 (a known, baseline address, which is the harder case). Five scenarios were used.

Table 17: Attack scenarios

Scenario	Description
brute_force	15 wrong passwords against one account, one to two seconds apart
spray	One common password tried against two accounts in three rounds
slow_low	Six wrong passwords 60 to 100 seconds apart, to stay below rate thresholds
stolen_credential	A correct password used once from a different source
token_burst	Ten login and logout cycles with the correct password, one to three seconds apart

In total 54 attack runs were executed: 27 from Kali and 27 from the laptop, with six runs per scenario and source (three for slow_low). Evidence of an attack is visible in Splunk; for example, a brute-force run from the Kali address appears as 17 failed logins in the keycloak index.

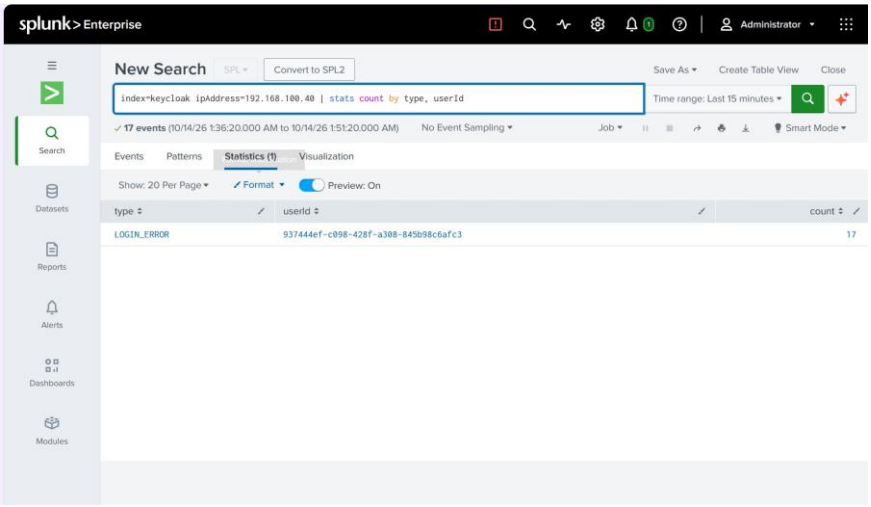


Figure 71: Brute-force evidence: 17 LOGIN_ERROR events from the Kali address

11. Source Code Explained

The project consists of eight Python files. This chapter says what each file is for, how it is run, and how it works, in the order in which they are used. The complete listings are in Appendix A. All files are stored in C:\catavo-ml on the laptop, except attacks.py, which runs on Kali (and on the laptop for the second attack source).

Table 18: Overview of the project files

File	Role in the pipeline	Reads	Writes
features.py	First version of the feature builder (five features). Superseded by features2.py.	keycloak_normal.csv	features.csv, baseline_ips.json
features2.py	Final data preparation: two normal batches, hold-out split, four model features.	keycloak_all.csv	features.csv, baseline_ips.json, keycloak_normal_split.csv
catavo_ml.py	Defines and trains the autoencoder; also the scoring functions used by every other script.	features.csv	model.pt, scaler.json, threshold.json, loss_history.csv, training_loss.png
baseline_if.py	Isolation Forest baseline trained on the same data.	features.csv, scaler.json	if_model.joblib, if_threshold.json
soar.py	Response actions: Keycloak session revoke, RHEL firewall block, pfSense block.	(network)	(network)
monitor.py	Live detection loop: polls Splunk, scores logins, triggers the response.	Splunk, model files	monitor_log.csv
attacks.py	Five attack scenarios that generate the test traffic.	(Keycloak)	attack_runs.csv
evaluate.py	Offline comparison of all detectors on the attack runs and on normal data.	monitor_log.csv, attack_runs_*.csv, keycloak_normal_split.csv	eval_summary.csv, eval_runs.csv, eval_output.txt

11.1 features.py (first version, superseded)

Purpose. Turns the raw Keycloak events exported from Splunk into one row of numbers per login. It was the first feature builder and used five features; it is kept for reference and its output files were archived in data\v1 when features2.py was introduced.

- Reads the CSV, converts the _time column to Indian Standard Time and drops the manual test logins made before 13 October 2026, 20:00.
- Separates LOGIN, LOGIN_ERROR and LOGOUT events. Each session's logout time is looked up by sessionId.
- login_hour: hour of the day counted from 04:00 (the quietest hour), so late-evening and after-midnight logins stay close together on the scale.
- session_duration: logout time minus login time of the same session, in seconds.
- failed_before_login: number of failed logins of the same user in the five minutes before the login.

- new_ip: 1 if the source address is not one of the addresses seen for that user, else 0; the known addresses are saved in baseline_ips.json.
- login_gap: seconds since the same user's previous login (the median is used for the first login).
- Writes features.csv and prints a statistics table.

11.2 features2.py (final data preparation)

Purpose. Builds the final training table and the hold-out set from the complete Splunk export keycloak_all.csv. It is run once with: python features2.py.

- **Backup.** First copies the old files (features, models, evaluation output) into data\v1, so nothing is lost when the data change.
- **Cleaning.** Keeps only LOGIN, LOGOUT and LOGIN_ERROR events that belong to a known user and come from one of the two normal addresses (192.168.100.50 for user1 and 192.168.100.70 for user2). Attack traffic from Kali is therefore excluded from the normal data.
- **Two batches.** WINDOWS defines B1 (13 Oct 20:00 to 14 Oct 01:00, evening) and B2 (14 Oct 13:19 to 15:00, daytime).
- **Hold-out.** In B2 the last 40 logins of each user (HOLD = 40) are tagged "holdout" and are never used for training; they give an honest false-positive rate.
- **Features.** Uses the same definitions as features.py but computes them separately for each batch, so that a gap never spans the break between batches. new_ip is computed against the single allowed address of each user.
- **Outputs.** features.csv (training rows only: 720), baseline_ips.json, and keycloak_normal_split.csv (all clean normal events with the columns batch and part). It prints the number of events kept, the statistics of the training features and the number of hold-out logins.

11.3 catavo_ml.py (the autoencoder)

Purpose. Defines the neural network, trains it, and provides the functions load_model(), prep(), errors() and score() that monitor.py and evaluate.py import. Run: python catavo_ml.py --epochs 1000.

Table 19: Elements of catavo_ml.py

Element	What it does
FEATURES	The four inputs of the model: session_duration, failed_before_login, new_ip, login_gap. login_hour is still recorded in the logs but is not used, because with only two time-of-day windows of lab data it caused false alarms on normal daytime logins.
prep(df)	Selects the four columns and applies log(1+x) to login_gap so that a rare very long gap does not dominate.
class AE	The network 4-3-2-3-4: an encoder (4 to 3 to 2 values, Tanh activation) squeezes each login into two numbers and a decoder (2 to 3 to 4) rebuilds the four inputs.
errors(model, x)	Returns the mean squared difference between the input and its reconstruction for each row: the anomaly score.
load_model()	Loads model.pt, the scaling values (scaler.json) and the threshold (threshold.json).
score(df)	Convenience function: scales a table of logins and returns the errors and the threshold.

Element	What it does
main block	Sets the random seed 42, shuffles the data and splits it 80/20, scales each feature to 0..1 using the training minimum and maximum (a constant feature such as new_ip gets span 1), trains with Adam (learning rate 0.01) on the mean-squared error for the chosen number of epochs, prints progress ten times, sets the threshold at the 99th percentile of the training errors, prints the validation false-positive rate and saves the model, scaler, threshold, loss history and the loss chart.

Because the model is trained only on normal logins, it learns to reproduce normal combinations of the four features. A login with a new IP address or many failed attempts cannot be rebuilt accurately, so its error is large and exceeds the threshold.

11.4 baseline_if.py (Isolation Forest baseline)

Purpose. Trains a standard anomaly detector from scikit-learn for comparison. It uses exactly the same four features, scaling and 80/20 split as the autoencoder (note that the comment at the top of the file still says "5 features" from the earlier version; the code uses four).

- IsolationForest(n_estimators=200, random_state=42) builds 200 random trees; points that are isolated in few splits get a high anomaly score.
- The score is the negative of scikit-learn's score_samples, so that higher means more anomalous, as for the autoencoder.
- The threshold is the 99th percentile of the training scores (0.6628), the same rule as for the autoencoder.
- Saves if_model.joblib and if_threshold.json and provides score_if(df) for the evaluation.

11.5 soar.py (automated response)

Purpose. Contains the three containment actions and a small command line. It is used by monitor.py and can also be run by hand: python soar.py status | block <ip> <userId> | unblock <ip>.

Table 20: Functions of soar.py

Function	What it does
secret(env, prompt)	Returns a password from an environment variable or asks for it once at a hidden prompt and caches it in memory. No password is stored in the code.
ssh_run(host, user, pw, cmd)	Opens an SSH connection with paramiko, runs one command, returns its output and closes the connection.
pf(cmd), rhel(cmd)	Run a command on pfSense (user root) or on the RHEL server (limited user soar).
kc_token()	Gets an admin access token from the Keycloak master realm using the administrator labadmin.
revoke_sessions(user_id)	Calls the Keycloak Admin API POST /admin/realms/catavo/users/{id}/logout; the answer 204 means all sessions of the user were ended.
block_rhel(ip), unblock_rhel(ip)	Add or remove the firewalld rich rule: rule family="ipv4" source address="<ip>" drop, with firewall-cmd on the RHEL server.
block_pfsense(ip), unblock_pfsense(ip)	Add or remove the address in the pfSense table blocked_ips with pfctl; pfctl -k also kills the existing connections of that address.

Function	What it does
check_ip(ip)	Validates the address and refuses to block the protected addresses: the firewall, the domain controller, Keycloak, Splunk and the laptop (192.168.100.1, .10, .20, .30, .70).
respond(ip, user_id)	Runs the three actions in order (Keycloak revoke, RHEL block, pfSense block), measures the time of each and returns the timings.

The RHEL rule is added at run time only, so it disappears when the firewall is reloaded or the server restarts; this is deliberate for a lab, because a stale block cannot lock out a user permanently.

11.6 monitor.py (live detection loop)

Purpose. Runs continuously on the laptop: reads new Keycloak events from Splunk, scores every login attempt with the autoencoder and, when the score is above the threshold, calls soar.py. Run: python monitor.py (dry run) or python monitor.py --act.

Table 21: Elements of monitor.py

Element	What it does
class Features	Keeps the small amount of state needed per user and builds the feature row of one event: failed attempts in the last 300 seconds, the new_ip flag against baseline_ips.json, the gap since the previous login and the hour. The session duration is not known at login time, so the training median is used. login_hour is computed and logged but is not given to the model.
splunk(), fetch()	Send a search to the Splunk REST interface on port 8089 and return the events with _indextime greater than a given value: LOGIN and LOGIN_ERROR events of realm catavo.
parse()	Converts one Splunk result into a Python record with time, type, user and IP.
respond()	For protected addresses only the Keycloak session is revoked; for any other address soar.respond() applies all three actions.
main()	Reads the options (--act, --poll seconds, --auto-unblock seconds), loads the model, baseline addresses and medians, asks for the credentials at hidden prompts, remembers the newest indexed event so that history is ignored at start-up, and then polls every three seconds.
scoring loop	For each new event: build features, scale, compute the error, compare with the threshold. If flagged and not already handled for that address and user: in dry run print "would_respond", in act mode run the response, record the timings and the event-to-done delay. Every event is appended to monitor_log.csv.
--auto-unblock N	Starts a timer that removes the block after N seconds (used for repeated latency tests).

The column event_to_done_s in monitor_log.csv is the end-to-end delay reported in Chapter 13; the individual action times are stored in keycloak_revoke_s, rhel_firewall_s and pfsense_block_s.

11.7 attacks.py (attack simulator)

Purpose. Generates the attack traffic of the evaluation. It uses only the Python standard library, so it runs on Kali and on the laptop. All attacks go to the catavo-sim client of realm catavo through the same token endpoint as normal logins.

Table 22: Attack scenarios implemented in attacks.py

Scenario	What the function does
brute_force	15 wrong passwords (guess0 to guess14) against one user, 1 to 2 seconds apart.
spray	Three rounds in which one common password is tried against user1 and user2, 3 to 6 seconds apart, with 20 seconds between rounds.
slow_low	6 wrong passwords, 60 to 100 seconds apart, to stay under rate-based rules.
stolen_credential	One login with the correct password from a different source, a pause of 10 to 40 seconds and a logout.
token_burst	10 login and logout cycles with the correct password, 1 to 3 seconds apart.

Helper functions: nap() sleeps for a random time; post(), login() and logout() call the Keycloak token and logout endpoints; the main block reads the arguments, asks for the correct password only when a scenario needs it, gives each run an identifier (scenario, host name, time stamp, number) and appends its start and end time to attack_runs.csv. These time stamps let evaluate.py assign every logged event to the run that produced it.

11.8 evaluate.py (evaluation)

Purpose. Compares all detectors offline on the same data and prints the result tables. Run: python evaluate.py | Tee-Object C:\catavo-ml\data\eval_output.txt.

- **Labelling.** label_attacks() matches the events in monitor_log.csv to the runs in attack_runs_kali.csv and attack_runs_laptop.csv by source address and time window, and tags each event with its run, scenario and source.
- **Fresh state per run.** Every run is featurised on its own, so that one attack cannot influence the features of the next.
- **Normal data.** keycloak_normal_split.csv is featurised per batch; the training rows give the in-sample false-positive rate and the 80 hold-out logins the unbiased one.
- **Detectors.** flags_for() produces a yes/no decision for the autoencoder, three ablated versions (login gap, new_ip or failed-login count frozen to their training median), the Isolation Forest at the 99th and 95th percentile, Rule 1 (five or more failures from one IP within five minutes), Rule 2 (a login from an IP outside the allow-list) and their combination.
- **Reports.** Prints the overall rates (false positives, attempt recall, runs detected), the runs detected per scenario and source, the number of attempts until the first alert, and false positives by attempt type; saves eval_summary.csv and eval_runs.csv.

12. Evaluation and Results

12.1 Detection performance

A run counts as detected if at least one alert is raised for it. Table 8 summarises the four detectors on the 54 attack runs and on normal data.

Table 23: Overall results

Detector	Runs detected	Detection rate	False positives (in-sample)	False positives (hold-out)
Autoencoder (4 features)	45 / 54	83.3 %	0.9 %	2.4 % (2 of 82)
Splunk rules 1 OR 2	38 / 54	70.4 %	0 %	0 %
Isolation Forest, 95th pct	32 / 54	59.3 %	4.9 %	8.5 %
Isolation Forest, 99th pct	0 / 54	0 %	-	-

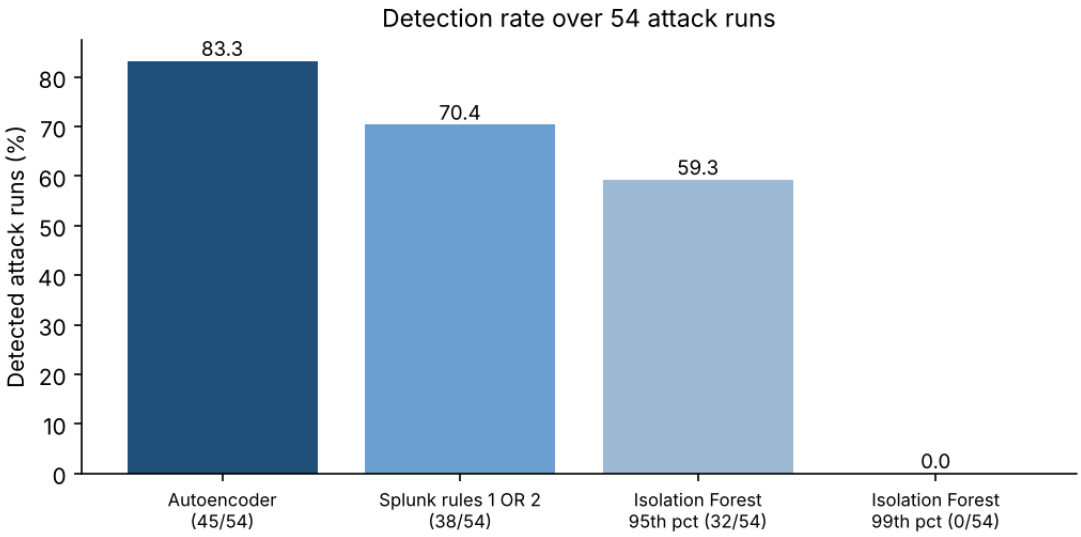


Figure 72: Detection rate of the four detectors

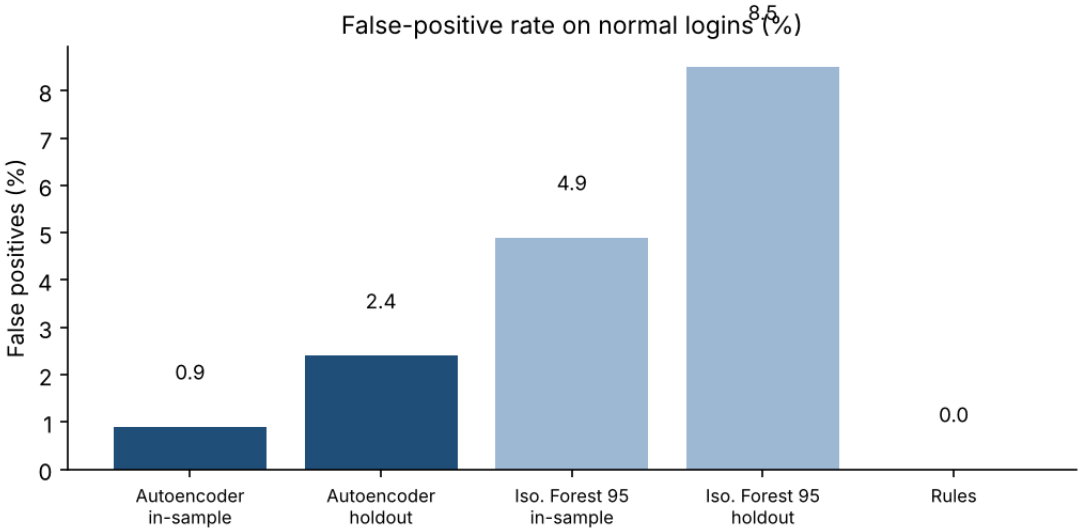


Figure 73: False-positive rate on normal logins

The autoencoder detects the most attack runs; at the attempt level its recall is 82.6 %. The median number of attempts needed to raise the first alert is one. On the harder case, attacks from the known laptop address, it detects 18 of 27 runs against 12 of 27 for the rules. The Isolation Forest at the strict threshold detects nothing, because new_ip never varies in the training data and its scores for brute-force attempts (about 0.654) stay just below the threshold of 0.6628; a looser threshold detects more but at a higher false-positive rate.

12.2 Per-scenario results

Table 24: Autoencoder detections per scenario and source

Scenario	From Kali	From laptop (known IP)
brute_force	6 / 6	6 / 6
spray	6 / 6	6 / 6
token_burst	6 / 6	6 / 6
stolen_credential	6 / 6	0 / 6
slow_low	3 / 3	0 / 3

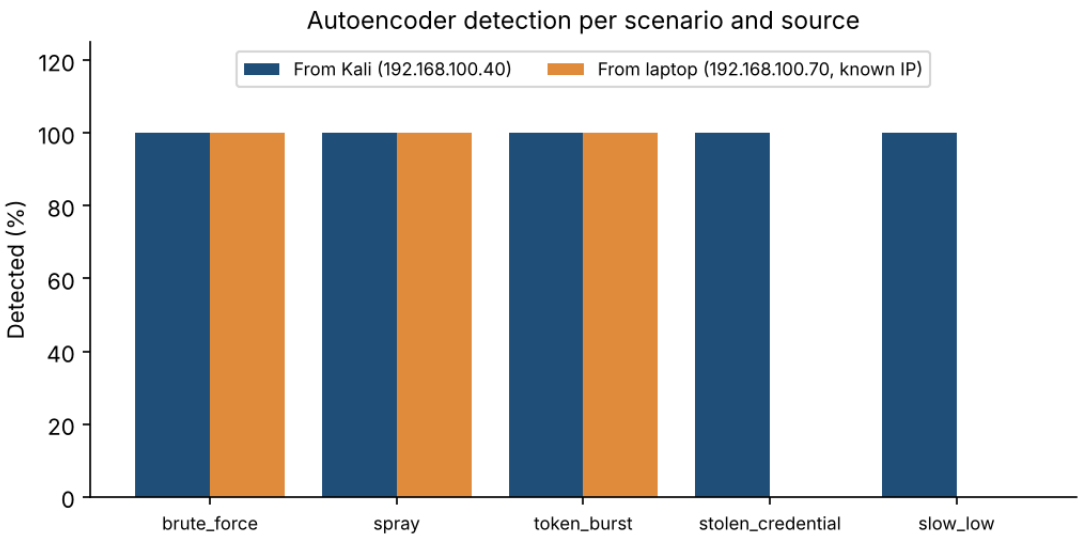


Figure 74: Autoencoder detection per scenario and source

12.3 Feature ablation

To see which features matter, each was frozen to a constant in turn. The new-IP signal is the strongest, followed by the login gap and the failed-login count.

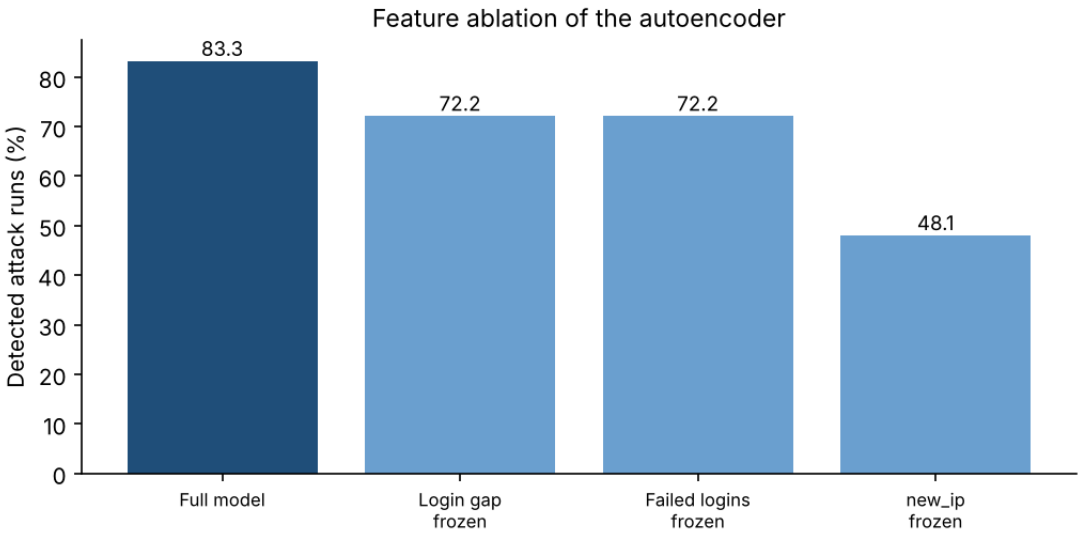


Figure 75: Autoencoder detection when one feature is frozen

12.4 Effect of removing the hour of day

Table 25: Five-feature and four-feature models

Model	Detection	Hold-out false positives
Five features (with login hour), threshold 0.07598	48 / 54 (88.9 %)	5 of 82 (6.1 %)
Four features (final), threshold 0.02303	45 / 54 (83.3 %)	2 of 82 (2.4 %)

Removing the hour reduced false alarms on unseen normal daytime logins at the price of three detected runs, which is the better trade-off for a system that takes automatic action. The five-feature model is kept as an ablation.

13. Response Time

The end-to-end delay was measured with ten single wrong-password attempts from Kali, 45 seconds apart, with the passwords entered before the monitor started so that typing time is not counted. The delay runs from the Keycloak event to the completed response.

Table 26: End-to-end latency over 10 trials

Statistic	Seconds
Minimum	4.4
Median	6.5
Mean	7.2
Maximum	11.9
Response actions only	1.3 to 4.6

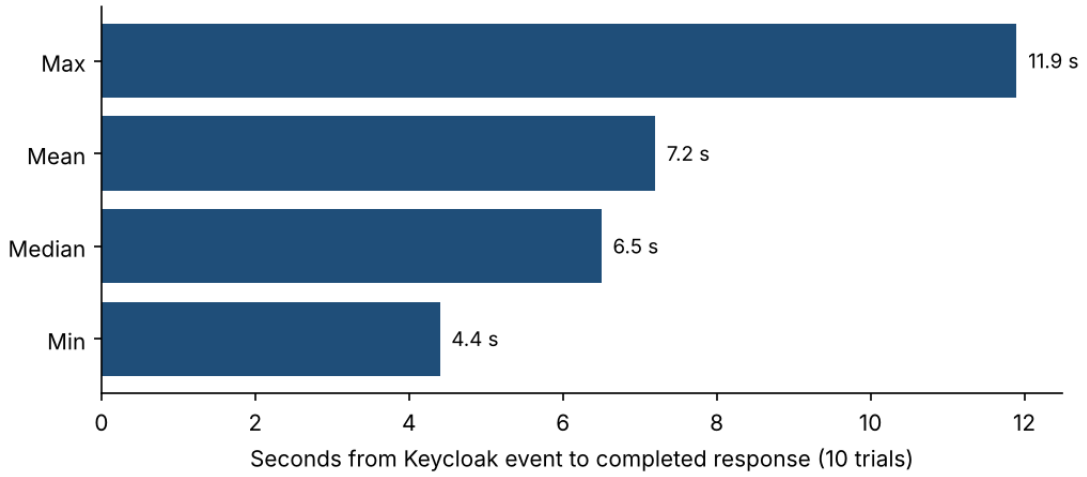


Figure 76: Latency from event to completed response

Most of the delay is Splunk indexing plus the three-second polling interval; the three response actions themselves take 1.3 to 4.6 seconds. The earlier design target of under 50 ms for the model decision is met by the model alone (a forward pass on four features) but not by the complete event-to-response loop, which is bounded by the SIEM path.

14. Findings

- An autoencoder trained only on normal logins detected 83.3 % of simulated attack runs with a hold-out false-positive rate of 2.4 %, outperforming both baselines on this lab.
- The model adds value beyond rules: it caught 18 of 27 attacks from a known address where the IP rule is blind (12 of 27 for all rules).
- Feature choice mattered more than training length: the model had converged after about 600 epochs, and removing the hour of day, not training longer, fixed the false positives.
- The new-IP feature behaves almost like a hard rule, and is the main reason for the detections from Kali.
- A network firewall alone cannot contain a peer on the same segment, so host-level blocking is needed in addition.
- The loop from event to response takes about seven seconds on average, dominated by SIEM latency rather than computation.

15. Limitations

- Sample sizes are small: three to six runs per scenario and source, and 82 attempts in the hold-out set.
- The attacks were written by the project team and may not represent real adversaries.
- Only two users were modelled, with narrow time-of-day windows and a regular cadence.
- The stolen-credential and slow-low scenarios from the known laptop address were not detected by any method.
- The Isolation Forest baseline is weak partly because its training data contain no variation in new_ip.
- Results come from a single laptop-hosted lab; the response engine runs in dry-run mode unless explicitly enabled.
- Active Directory account lockout and vulnerability scanning are not implemented.

16. Future Work

- **Active Directory lockout.** Extend the response to disable or lock the account in the domain controller (currently the implemented responses are the Keycloak session revoke, the RHEL firewall block and the pfSense block).
- **Vulnerability orchestration.** Add the automated scanning and orchestration half of CAT-AVO and feed the exposure of a host into its trust score.
- Collect more users and longer baselines, add contextual features such as device and geography, and evaluate with more varied and adaptive attacks.
- Reduce the loop delay with a streaming path from Keycloak instead of SIEM polling.
- Explore per-user models and sequence models.

17. Conclusion

CAT-AVO demonstrates, in a reproducible isolated lab, that a small unsupervised autoencoder can score post-authentication behaviour from identity-provider events and drive an automatic response across Keycloak, a host firewall and pfSense. On 54 simulated attack runs it detected 83.3 % at a 2.4 % hold-out false-positive rate, ahead of correlation rules and Isolation Forest, and it contained attackers in about seven seconds on average. The limitations are stated openly: the evaluation is small and synthetic, and some known-address attacks remain invisible to every method. The platform, code and data pipeline provide a base for the planned extensions.

Appendix A: Source Code Listings

The listings below are the final versions of the project files, reproduced unchanged. Explanations are in Chapter 11.

features.py: first feature builder (superseded)

```
import json, sys
import pandas as pd

SRC = sys.argv[1] if len(sys.argv) > 1 else r"C:\catavo-ml\data\keycloak_normal.csv"
OUT = sys.argv[2] if len(sys.argv) > 2 else r"C:\catavo-ml\data\features.csv"
BASE = sys.argv[3] if len(sys.argv) > 3 else r"C:\catavo-ml\data\baseline_ips.json"
START = "2026-10-13 20:00"   # skip the manual tests from Oct 12

d = pd.read_csv(SRC)
d["t"] = pd.to_datetime(d["_time"], utc=True).dt.tz_convert("Asia/Kolkata")
d = d[d["t"] >= pd.Timestamp(START, tz="Asia/Kolkata")].sort_values("t").reset_index(drop=True)

logins = d[d.type == "LOGIN"].copy()
errors = d[d.type == "LOGIN_ERROR"]
lo = d[d.type == "LOGOUT"].drop_duplicates("sessionId").set_index("sessionId")["t"]

# day starts at 04:00 (quietest hour) so late-evening and after-midnight logins stay close together
logins["login_hour"] = (logins.t.dt.hour + logins.t.dt.minute / 60 - 4) % 24
logins["session_duration"] = (logins.sessionId.map(lo) - logins.t).dt.total_seconds()

def fails(r):
    e = errors[(errors.userId == r.userId) & (errors.t < r.t) & (errors.t >= r.t - pd.Timedelta(minutes=5))]
    return len(e)
logins["failed_before_login"] = logins.apply(fails, axis=1)

baseline = logins.groupby("userId").ipAddress.agg(lambda s: sorted(set(s))).to_dict()
logins["new_ip"] = [int(r.ipAddress not in baseline[r.userId]) for r in logins.itertuples()]

gap = logins.groupby("userId").t.diff().dt.total_seconds()
logins["login_gap"] = gap.fillna(gap.median())

cols = ["login_hour", "session_duration", "failed_before_login", "new_ip", "login_gap"]
out = logins.dropna(subset=cols)[["userId", "t"] + cols]
out.to_csv(OUT, index=False)
json.dump(baseline, open(BASE, "w"), indent=1)
print("rows:", len(out), "dropped (no logout):", len(logins) - len(out))
print(out[cols].describe().round(2).T[["mean", "std", "min", "max"]])
print(baseline)
```

features2.py: final data preparation

```
r"""CAT-AVO features v2: two normal batches (evening 13 Oct and daytime 14 Oct).
Run:  python features2.py     (needs data\keycloak_all.csv, the full Splunk export)
Writes: features.csv (training rows), baseline_ips.json, keycloak_normal_split.csv (all clean normal events,
tagged
batch and part = train / holdout). Old files are copied to data\v1 first.
Holdout = the last HOLD logins of each user in batch B2; they are never used for training."""
import json, os, shutil
import pandas as pd

DATA = os.environ.get("CATAVO_DATA", r"C:\catavo-ml\data")
P = lambda n: os.path.join(DATA, n)
IST = "Asia/Kolkata"
WINDOWS = [("B1", "2026-10-13 20:00", "2026-10-14 01:00"),
           ("B2", "2026-10-14 13:19", "2026-10-14 15:00")]
HOLD = 40
ALLOWED = {"937444ef-c098-428f-a308-845b98c6afc3": "192.168.100.50",    # user1
           "a2bff448-a23d-4de0-bfa8-6e97263d9815": "192.168.100.70"}    # user2
COLS = ["login_hour", "session_duration", "failed_before_login", "new_ip", "login_gap"]

# 1. keep the old (v1) files
v1 = P("v1")
if not os.path.isdir(v1):
    os.makedirs(v1)
    for f in ("features.csv", "baseline_ips.json", "model.pt", "scaler.json", "threshold.json", "if_model.joblib",
              "if_threshold.json", "loss_history.csv", "training_loss.png", "eval_summary.csv", "eval_runs.csv",
              "eval_output.txt", "keycloak_normal.csv"):
        if os.path.exists(P(f)):
            shutil.copy(P(f), v1)
    print("old files copied to", v1)

# 2. read, keep only the clean normal windows and the two normal source IPs
d = pd.read_csv(P("keycloak_all.csv"))
d["t"] = pd.to_datetime(d["_time"], utc=True, format="ISO8601").dt.tz_convert(IST)
d = d[d.userId.notna() & d.type.isin(["LOGIN", "LOGOUT", "LOGIN_ERROR"])].copy()
ip_ok = d.ipAddress.isin(ALLOWED.values())
d = d[ip_ok].sort_values("t").reset_index(drop=True)
print("events kept per window and type:")

parts = []
for name, a, b in WINDOWS:
    w = d[(d.t >= pd.Timestamp(a, tz=IST)) & (d.t < pd.Timestamp(b, tz=IST))].copy()
    w["batch"] = name
    w["part"] = "train"
    if name == "B2":                                    # chronological holdout: last HOLD logins per user
        for u, g in w[w.type == "LOGIN"].groupby("userId"):
            ordered = g.sort_values("t")
            if len(ordered) > HOLD:
                cut = ordered.iloc[len(ordered) - HOLD].t
                w.loc[(w.userId == u) & (w.t >= cut), "part"] = "holdout"
    print(name, w.groupby(["type", "part"]).size().to_dict())
    parts.append(w)
allev = pd.concat(parts, ignore_index=True)
allev.drop(columns=["t"]).to_csv(P("keycloak_normal_split.csv"), index=False)

# 3. training features (same definitions as features.py), per batch so a gap never spans the break between batches
rows = []
for name, w in allev.groupby("batch"):
    logins = w[w.type == "LOGIN"].copy()
    errors = w[w.type == "LOGIN_ERROR"]
    lo = w[w.type == "LOGOUT"].drop_duplicates("sessionId").set_index("sessionId")["t"]
    logins["login_hour"] = (logins.t.dt.hour + logins.t.dt.minute / 60 - 4) % 24
    logins["session_duration"] = (logins.sessionId.map(lo) - logins.t).dt.total_seconds()
    logins["failed_before_login"] = [
        len(errors[(errors.userId == r.userId) & (errors.t < r.t) & (errors.t >= r.t - pd.Timedelta(minutes=5))])
        for r in logins.itertuples()]
    logins["new_ip"] = [int(r.ipAddress != ALLOWED[r.userId]) for r in logins.itertuples()]
    logins["login_gap"] = logins.groupby("userId").t.diff().dt.total_seconds()
    rows.append(logins)
f = pd.concat(rows, ignore_index=True)
f["login_gap"] = f.login_gap.fillna(f.login_gap.median())      # first login of a batch has no previous login
f = f[f.part == "train"].dropna(subset=COLS)
f[["userId", "t"] + COLS].to_csv(P("features.csv"), index=False)
json.dump({u: [ip] for u, ip in ALLOWED.items()}, open(P("baseline_ips.json"), "w"), indent=1)

print("\ntraining rows:", len(f), " per batch:", f.groupby("batch").size().to_dict())
print(f[COLS].describe().round(2).T[["mean", "std", "min", "max"]])
print("holdout logins:", int(((allev.part == "holdout") & (allev.type == "LOGIN")).sum()))
```

catavo_ml.py: autoencoder training and scoring

```
"""CAT-AVO autoencoder (4-3-2-3-4). Run:  python catavo_ml.py   to train.
Other scripts can `from catavo_ml import load_model, score`."""
import argparse, json, os
import numpy as np
import pandas as pd
import torch
from torch import nn

DATA = r"C:\catavo-ml\data"
# login_hour is still logged but NOT used by the model: with only two time-of-day windows of lab data it caused
# false alarms on normal daytime logins (holdout FPR 6.1% vs 0%) without improving attack detection.
FEATURES = ["session_duration", "failed_before_login", "new_ip", "login_gap"]
N = len(FEATURES)



def prep(df):
    x = df[FEATURES].astype(float).copy()
    x["login_gap"] = np.log1p(x["login_gap"])      # tame the rare very long gap
    return x.values



class AE(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(N, 3), nn.Tanh(), nn.Linear(3, 2), nn.Tanh())
        self.dec = nn.Sequential(nn.Linear(2, 3), nn.Tanh(), nn.Linear(3, N))

    def forward(self, x):
        return self.dec(self.enc(x))



def errors(model, x):
    with torch.no_grad():
        t = torch.tensor(x, dtype=torch.float32)
        return ((model(t) - t) ** 2).mean(dim=1).numpy()



def load_model():
    m = AE()
    m.load_state_dict(torch.load(os.path.join(DATA, "model.pt")))
    m.eval()
    sc = json.load(open(os.path.join(DATA, "scaler.json")))
    th = json.load(open(os.path.join(DATA, "threshold.json")))["threshold"]
    return m, np.array(sc["lo"]), np.array(sc["span"]), th



def score(df):
    """Return reconstruction error for each row of a feature DataFrame."""
    m, lo, span, th = load_model()
    return errors(m, (prep(df) - lo) / span), th



if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--epochs", type=int, default=500)
    EPOCHS = ap.parse_args().epochs
    torch.manual_seed(42)
    np.random.seed(42)
    df = pd.read_csv(os.path.join(DATA, "features.csv"))
    x = prep(df)
    idx = np.random.permutation(len(x))
    cut = int(0.8 * len(x))
    tr, va = x[idx[:cut]], x[idx[cut:]]

    lo = tr.min(axis=0)
    span = tr.max(axis=0) - lo
    span[span == 0] = 1.0                          # constant feature (new_ip = 0)
    tr_s, va_s = (tr - lo) / span, (va - lo) / span

    model = AE()
    opt = torch.optim.Adam(model.parameters(), lr=0.01)
    loss_fn = nn.MSELoss()
    xt = torch.tensor(tr_s, dtype=torch.float32)
    xv = torch.tensor(va_s, dtype=torch.float32)
    hist = []
    for ep in range(1, EPOCHS + 1):
        opt.zero_grad()
        loss = loss_fn(model(xt), xt)
        loss.backward()
        opt.step()
        with torch.no_grad():
            vloss = loss_fn(model(xv), xv).item()
        hist.append((ep, loss.item(), vloss))
        if ep == 1 or ep % (EPOCHS // 10) == 0:
            print(f"epoch {ep:3d}  train {loss.item():.5f}  val {vloss:.5f}")
    model.eval()

    e_tr, e_va = errors(model, tr_s), errors(model, va_s)
    thr = float(np.percentile(e_tr, 99))
    print(f"threshold (99th pct of training error) = {thr:.5f}")
    print(f"validation false-positive rate = {(e_va > thr).mean() * 100:.1f}%  ({(e_va > thr).sum()} of
{len(e_va)})")
    print(f"error mean {e_tr.mean():.5f}  max {e_tr.max():.5f}")

    torch.save(model.state_dict(), os.path.join(DATA, "model.pt"))
    json.dump({"lo": lo.tolist(), "span": span.tolist()}, open(os.path.join(DATA, "scaler.json"), "w"))
    json.dump({"threshold": thr}, open(os.path.join(DATA, "threshold.json"), "w"))
    pd.DataFrame(hist, columns=["epoch", "train_loss", "val_loss"]).to_csv(os.path.join(DATA, "loss_history.csv"),
index=False)
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
h = pd.DataFrame(hist, columns=["epoch", "train", "val"])
plt.plot(h.epoch, h.train, label="train")
plt.plot(h.epoch, h.val, label="validation")
plt.yscale("log")
plt.xlabel("epoch")
plt.ylabel("MSE loss")
plt.title("Autoencoder training (4-3-2-3-4)")
plt.legend()
plt.savefig(os.path.join(DATA, "training_loss.png"), dpi=150, bbox_inches="tight")
print("saved model.pt, scaler.json, threshold.json, training_loss.png in", DATA)
```

baseline_if.py: Isolation Forest baseline

```
r"""Isolation Forest baseline on the same 5 features, scaling and split as the autoencoder.
Run:  python baseline_if.py      (needs data\features.csv and data\scaler.json from catavo_ml.py)"""
import json, os
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest

DATA = os.environ.get("CATAVO_DATA", r"C:\catavo-ml\data")
FEATURES = ["session_duration", "failed_before_login", "new_ip", "login_gap"]    # same 4 features as the
autoencoder



def prep(df):
    x = df[FEATURES].astype(float).copy()
    x["login_gap"] = np.log1p(x["login_gap"])
    return x.values



def load_scaler():
    sc = json.load(open(os.path.join(DATA, "scaler.json")))
    return np.array(sc["lo"]), np.array(sc["span"])



def score_if(df):
    """Return (anomaly score, threshold). Higher score = more anomalous; flag if score > threshold."""
    m = joblib.load(os.path.join(DATA, "if_model.joblib"))
    thr = json.load(open(os.path.join(DATA, "if_threshold.json")))["threshold"]
    lo, span = load_scaler()
    return -m.score_samples((prep(df) - lo) / span), thr



if __name__ == "__main__":
    np.random.seed(42)                          # same split as catavo_ml.py
    df = pd.read_csv(os.path.join(DATA, "features.csv"))
    x = prep(df)
    idx = np.random.permutation(len(x))
    cut = int(0.8 * len(x))
    lo, span = load_scaler()
    tr, va = (x[idx[:cut]] - lo) / span, (x[idx[cut:]] - lo) / span

    m = IsolationForest(n_estimators=200, random_state=42).fit(tr)
    s_tr, s_va = -m.score_samples(tr), -m.score_samples(va)
    thr = float(np.percentile(s_tr, 99))     # same rule as the autoencoder: 99th percentile of normal
    print(f"threshold (99th pct of training score) = {thr:.4f}")
    print(f"validation false-positive rate = {(s_va > thr).mean() * 100:.1f}%  ({(s_va > thr).sum()} of
{len(s_va)})")
    joblib.dump(m, os.path.join(DATA, "if_model.joblib"))
    json.dump({"threshold": thr}, open(os.path.join(DATA, "if_threshold.json"), "w"))
    print("saved if_model.joblib and if_threshold.json in", DATA)
```

soar.py: automated response

```
r"""CAT-AVO SOAR response actions.
  python soar.py status                       check the three connections
  python soar.py block   <attacker_ip> <userId>   revoke Keycloak sessions + block IP on RHEL and pfSense
  python soar.py unblock <attacker_ip>        remove both IP blocks
Passwords come from the environment (SOAR_KC_PW, SOAR_PF_PW, SOAR_RHEL_PW) or are asked once."""
import getpass, ipaddress, os, sys, time
import paramiko
import requests

KC = "http://192.168.100.20:8080"
REALM = "catavo"
PF_HOST, RHEL_HOST = "192.168.100.1", "192.168.100.20"
PROTECTED = {"192.168.100.1", "192.168.100.10", "192.168.100.20", "192.168.100.30", "192.168.100.70"}
_cache = {}



def secret(env, prompt):
    if env not in _cache:
        _cache[env] = os.environ.get(env) or getpass.getpass(prompt)
    return _cache[env]



def ssh_run(host, user, pw, cmd):
    c = paramiko.SSHClient()
    c.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    c.connect(host, username=user, password=pw, look_for_keys=False, allow_agent=False, timeout=10)
    try:
        _, o, e = c.exec_command(cmd, timeout=20)
        out, err = o.read().decode().strip(), e.read().decode().strip()
    finally:
        c.close()
    return (out + " " + err).strip()



def pf(cmd):
    return ssh_run(PF_HOST, "root", secret("SOAR_PF_PW", "pfSense root password: "), cmd)



def rhel(cmd):
    return ssh_run(RHEL_HOST, "soar", secret("SOAR_RHEL_PW", "RHEL soar password: "), cmd)



def kc_token():
    r = requests.post(f"{KC}/realms/master/protocol/openid-connect/token", timeout=10, data={
        "client_id": "admin-cli", "grant_type": "password", "username": "labadmin",
        "password": secret("SOAR_KC_PW", "Keycloak labadmin password: ")})
    r.raise_for_status()
    return r.json()["access_token"]



def revoke_sessions(user_id):
    r = requests.post(f"{KC}/admin/realms/{REALM}/users/{user_id}/logout", timeout=10,
                      headers={"Authorization": "Bearer " + kc_token()})
    return r.status_code == 204



def rule(ip):
    return f"rule family=\"ipv4\" source address=\"{ip}\" drop"



def block_rhel(ip):
    return "success" in rhel(f"sudo firewall-cmd --add-rich-rule='{rule(ip)}'")



def unblock_rhel(ip):
    return "success" in rhel(f"sudo firewall-cmd --remove-rich-rule='{rule(ip)}'")



def block_pfsense(ip):
    t = pf(f"pfctl -t blocked_ips -T add {ip}; pfctl -k {ip}")
    return "1/1 addresses added" in t or "0/1" in t



def unblock_pfsense(ip):
    return "1/1 addresses deleted" in pf(f"pfctl -t blocked_ips -T delete {ip}")



def check_ip(ip):
    ipaddress.ip_address(ip)
    if ip in PROTECTED:
        sys.exit(f"Refusing to block {ip}: it is lab infrastructure or the SOAR host.")



def respond(ip, user_id=None):
    """Full response. Returns per-action seconds (None = failed) and total."""
    check_ip(ip)
    t0 = time.perf_counter()
    res = {}
    steps = ([("keycloak_revoke", lambda: revoke_sessions(user_id))] if user_id else []) + [
        ("rhel_firewall", lambda: block_rhel(ip)), ("pfsense_block", lambda: block_pfsense(ip))]
    for name, fn in steps:
        s = time.perf_counter()
        try:
            ok = fn()
```

```
        except Exception as ex:
            print(f"  {name}: ERROR {ex}")
            ok = False
        res[name] = round(time.perf_counter() - s, 2) if ok else None
        print(f"  {name}: {'OK' if ok else 'FAILED'}  ({time.perf_counter() - t0:.2f}s since start)")
    res["total"] = round(time.perf_counter() - t0, 2)
    return res


if __name__ == "__main__":
    a = sys.argv[1:]
    if a[:1] == ["status"]:
        print("keycloak admin token:", "OK" if kc_token() else "FAILED")
        print("pfSense blocked_ips :", pf("pfctl -t blocked_ips -T show") or "(empty)")
        print("RHEL rich rules     :", rhel("sudo firewall-cmd --list-rich-rules") or "(none)")
    elif a[:1] == ["block"] and len(a) == 3:
        print(respond(a[1], a[2]))
    elif a[:1] == ["unblock"] and len(a) == 2:
        check_ip(a[1])
        print("rhel:", unblock_rhel(a[1]), " pfsense:", unblock_pfsense(a[1]))
    else:
        print(__doc__)
```

monitor.py: live detection loop

```
r"""CAT-AVO monitor: polls Splunk for Keycloak login events, scores every authentication attempt
with the autoencoder and (with --act) responds through soar.py.
  python monitor.py          dry run: score and log only, no response
  python monitor.py --act    real response: revoke sessions, block IP on RHEL and pfSense
Stop with Ctrl+C.  Log: data\monitor_log.csv"""
import argparse, csv, json, os, threading, time
from collections import defaultdict, deque
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

import numpy as np
import pandas as pd
import requests

import catavo_ml
import soar

DATA = os.environ.get("CATAVO_DATA", r"C:\catavo-ml\data")
SPLUNK = "https://192.168.100.30:8089"
IST = ZoneInfo("Asia/Kolkata")
LOG_COLS = ["event_time", "type", "userId", "ip", "score", "threshold", "flagged", "action",
            "keycloak_revoke_s", "rhel_firewall_s", "pfsense_block_s", "total_s", "event_to_done_s",
            "login_hour", "session_duration", "failed_before_login", "new_ip", "login_gap"]



class Features:
    """Builds the 5 features for one attempt from what has been seen so far."""

    def __init__(self, baseline, med_duration, med_gap):
        self.base, self.med_dur, self.med_gap = baseline, med_duration, med_gap
        self.errors = defaultdict(deque)
        self.last_login = {}

    def row(self, ev):
        t, u = ev["t"], ev["userId"]
        q = self.errors[u]
        while q and (t - q[0]).total_seconds() > 300:
            q.popleft()
        failed = sum(1 for x in q if x < t)
        local = t.astimezone(IST)
        hour = (local.hour + local.minute / 60 - 4) % 24
        new_ip = int(ev["ip"] not in self.base.get(u, []))
        gap = (t - self.last_login[u]).total_seconds() if u in self.last_login else self.med_gap
        if ev["type"] == "LOGIN_ERROR":
            q.append(t)
        else:
            self.last_login[u] = t
        return {"login_hour": hour, "session_duration": self.med_dur, "failed_before_login": failed,
                "new_ip": new_ip, "login_gap": gap}



def splunk(auth, query, earliest="-10m"):
    r = requests.post(f"{SPLUNK}/services/search/jobs/export", auth=auth, verify=False, timeout=30,
                      data={"search": query, "output_mode": "json", "earliest_time": earliest, "latest_time":
"now"})
    r.raise_for_status()
    out = []
    for line in r.text.splitlines():
        if line.strip():
            j = json.loads(line)
            if "result" in j:
                out.append(j["result"])
    return out



def fetch(auth, since):
    q = ("search index=keycloak realmName=catavo (type=LOGIN OR type=LOGIN_ERROR) "
         f"| eval it=_indextime, t=_time | where it>={since} | sort 0 +t "
         "| table t, it, type, userId, ipAddress")
    return splunk(auth, q)



def parse(r):
    return {"t": datetime.fromtimestamp(float(r["t"]), tz=timezone.utc), "it": float(r["it"]), "type": r["type"],
            "userId": r.get("userId"), "ip": r.get("ipAddress")}



def respond(ip, user_id):
    if ip in soar.PROTECTED:                        # never firewall lab infrastructure or the SOAR host
        s = time.perf_counter()
        ok = soar.revoke_sessions(user_id)
        return {"keycloak_revoke": round(time.perf_counter() - s, 2) if ok else None,
                "total": round(time.perf_counter() - s, 2)}
    return soar.respond(ip, user_id)



def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--act", action="store_true")
    ap.add_argument("--poll", type=float, default=3.0)
    ap.add_argument("--auto-unblock", type=float, default=0, help="test mode: undo the block this many seconds
after responding")
    args = ap.parse_args()
    requests.packages.urllib3.disable_warnings()

    model, lo, span, thr = catavo_ml.load_model()
    base = json.load(open(os.path.join(DATA, "baseline_ips.json")))
    ft = pd.read_csv(os.path.join(DATA, "features.csv"))
    feats = Features(base, float(ft.session_duration.median()), float(ft.login_gap.median()))
```

```
    user = os.environ.get("SOAR_SPLUNK_USER") or input("Splunk username [admin]: ").strip() or "admin"
    auth = (user, soar.secret("SOAR_SPLUNK_PW", f"Splunk password for {user}: "))
    if args.act:
        for env, p in (("SOAR_KC_PW", "Keycloak labadmin password: "), ("SOAR_PF_PW", "pfSense root password: "),
                       ("SOAR_RHEL_PW", "RHEL soar password: ")):
            soar.secret(env, p)

    since = float(splunk(auth, "search index=keycloak | stats max(_indextime) as m", "0")[0]["m"])
    path = os.path.join(DATA, "monitor_log.csv")
    new = not os.path.exists(path)
    logf = open(path, "a", newline="")
    log = csv.DictWriter(logf, LOG_COLS)
    if new:
        log.writeheader()
    print(f"monitor started ({'ACTING' if args.act else 'dry run'}), threshold {thr:.5f}. Ctrl+C to stop.")

    seen, handled, n = set(), set(), 0
    for r in fetch(auth, since):                  # events already there at startup are history: mark seen, do
not act
        seen.add((r["t"], r["type"], r.get("userId"), r.get("ipAddress")))
    while True:
        try:
            rows = fetch(auth, since)
        except Exception as ex:
            print("splunk error:", ex)
            time.sleep(args.poll)
            continue
        for r in rows:
            key = (r["t"], r["type"], r.get("userId"), r.get("ipAddress"))
            if key in seen:
                continue
            seen.add(key)
            since = max(since, float(r["it"]))
            ev = parse(r)
            if not ev["userId"]:
                continue                                  # unknown user name: only the Splunk rules cover this
            f = feats.row(ev)
            x = (catavo_ml.prep(pd.DataFrame([f])) - lo) / span
            score = float(catavo_ml.errors(model, x)[0])
            flagged = score > thr
            rec = {"event_time": ev["t"].isoformat(), "type": ev["type"], "userId": ev["userId"], "ip": ev["ip"],
                   "score": round(score, 5), "threshold": round(thr, 5), "flagged": int(flagged), "action": "",
**f}
            n += 1
            if flagged:
                pair = (ev["ip"], ev["userId"])
                if pair in handled:
                    rec["action"] = "already_handled"
                elif args.act:
                    res = respond(ev["ip"], ev["userId"])
                    done = datetime.now(timezone.utc)
                    handled.add(pair)
                    rec.update({"action": "responded", "keycloak_revoke_s": res.get("keycloak_revoke"),
                                "rhel_firewall_s": res.get("rhel_firewall"), "pfsense_block_s":
res.get("pfsense_block"),
                                "total_s": res.get("total"),
                                "event_to_done_s": round((done - ev["t"]).total_seconds(), 2)})
                    if args.auto_unblock and ev["ip"] not in soar.PROTECTED:
                        def undo(ip=ev["ip"], pair=pair):
                            try:
                                soar.unblock_rhel(ip)
                                soar.unblock_pfsense(ip)
                                print(f"  auto-unblocked {ip}")
                            except Exception as ex:
                                print("  auto-unblock error:", ex)
                            handled.discard(pair)
                        tm = threading.Timer(args.auto_unblock, undo)
                        tm.daemon = True
                        tm.start()
                else:
                    rec["action"] = "would_respond"
                print(f"[{ev['t'].astimezone(IST):%H:%M:%S}] ANOMALY {ev['type']} ip={ev['ip']} score={score:.4f}
-> {rec['action']}")
            elif n % 25 == 0:
                print(f"  {n} attempts scored, last score {score:.4f} (normal)")
            log.writerow(rec)
            logf.flush()
        time.sleep(args.poll)


if __name__ == "__main__":
    main()
```

attacks.py: attack simulator

```
#!/usr/bin/env python3
"""CAT-AVO attack simulator (lab only). Standard library only, runs on Kali or on the laptop.
  python3 attacks.py <scenario> [--user user1] [--runs 1] [--pause 120]
Scenarios: brute_force, spray, slow_low, stolen_credential, token_burst
Appends ground truth (run id, scenario, host, start, end) to attack_runs.csv in the current folder."""
import argparse, csv, getpass, json, os, random, socket, time
import urllib.error, urllib.parse, urllib.request
from datetime import datetime

KC = os.environ.get("CATAVO_KC", "http://192.168.100.20:8080/realms/catavo/protocol/openid-connect")
SPEED = float(os.environ.get("CATAVO_SPEED", "1"))      # >1 only for testing the script itself



def nap(a, b=None):
    time.sleep(random.uniform(a, b if b is not None else a) / SPEED)



def post(path, data):
    req = urllib.request.Request(f"{KC}/{path}", data=urllib.parse.urlencode(data).encode())
    try:
        with urllib.request.urlopen(req, timeout=10) as r:
            return r.status, json.loads(r.read() or b"{}")
    except urllib.error.HTTPError as e:
        return e.code, {}
    except Exception:
        return 0, {}



def login(user, pw):
    return post("token", {"client_id": "catavo-sim", "grant_type": "password",
                          "username": user, "password": pw, "scope": "openid"})



def logout(tok):
    post("logout", {"client_id": "catavo-sim", "refresh_token": tok})



def brute_force(user, pw):
    for i in range(15):
        login(user, f"guess{i}")
        nap(1, 2)



def spray(user, pw):
    for rnd in range(3):
        for u in ("user1", "user2"):
            login(u, "Winter2026!")
            nap(3, 6)
        nap(20)



def slow_low(user, pw):
    for i in range(6):
        login(user, f"slow{i}")
        nap(60, 100)



def stolen_credential(user, pw):
    code, tok = login(user, pw)
    print("  login status", code)
    nap(10, 40)
    if tok.get("refresh_token"):
        logout(tok["refresh_token"])



def token_burst(user, pw):
    for _ in range(10):
        code, tok = login(user, pw)
        if tok.get("refresh_token"):
            logout(tok["refresh_token"])
        nap(1, 3)



SCENARIOS = {f.__name__: f for f in (brute_force, spray, slow_low, stolen_credential, token_burst)}

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("scenario", choices=SCENARIOS)
    ap.add_argument("--user", default="user1")
    ap.add_argument("--runs", type=int, default=1)
    ap.add_argument("--pause", type=float, default=120, help="seconds between runs")
    a = ap.parse_args()
    pw = (os.environ.get("CATAVO_PW") or getpass.getpass(f"Correct password for {a.user}: ")) if a.scenario in
("stolen_credential", "token_burst") else ""
    new = not os.path.exists("attack_runs.csv")
    f = open("attack_runs.csv", "a", newline="")
    w = csv.writer(f)
    if new:
        w.writerow(["run_id", "scenario", "host", "user", "start", "end"])
    for n in range(1, a.runs + 1):
        rid = f"{a.scenario}-{socket.gethostname()}-{datetime.now():%m%d%H%M%S}-{n}"
        start = datetime.now().astimezone().isoformat(timespec="seconds")
        print(f"run {n}/{a.runs}  {rid}  started {start}")
        SCENARIOS[a.scenario](a.user, pw)
        end = datetime.now().astimezone().isoformat(timespec="seconds")
        w.writerow([rid, a.scenario, socket.gethostname(), a.user, start, end])
        f.flush()
```

```
        print(f"  finished {end}")
        if n < a.runs:
            nap(a.pause)
```

evaluate.py: evaluation

```
r"""CAT-AVO evaluation (Task 21). Offline: scores the attack runs and the normal traffic with the
autoencoder, the Isolation Forest and the two Splunk rules.
  python evaluate.py [--normal2 C:\catavo-ml\data\keycloak_normal2.csv]
Needs in the data folder: keycloak_normal.csv, monitor_log.csv, attack_runs_kali.csv, attack_runs_laptop.csv,
features.csv, baseline_ips.json and the trained models."""
import argparse, json, os
import numpy as np
import pandas as pd

import baseline_if
import catavo_ml
import monitor

DATA = os.environ.get("CATAVO_DATA", r"C:\catavo-ml\data")
START = pd.Timestamp("2026-10-13 20:00", tz="Asia/Kolkata")
ALLOWED = {"192.168.100.50", "192.168.100.70"}
HOST_IP = {"catavo-kali": "192.168.100.40", "YashSahu": "192.168.100.70"}
FEATS = ["login_hour", "session_duration", "failed_before_login", "new_ip", "login_gap"]
P = lambda n: os.path.join(DATA, n)



def clean(d, after=None, keep=()):
    d = d.copy()
    d["t"] = pd.to_datetime(d["_time"], utc=True, format="ISO8601")
    d = d[d.type.isin(["LOGIN", "LOGIN_ERROR"]) & d.userId.notna()].rename(columns={"ipAddress": "ip"})
    if after is not None:
        d = d[d.t >= after]
    return d.sort_values("t")[["t", "type", "userId", "ip", *keep]].reset_index(drop=True)



def load_events(path, after=None):
    return clean(pd.read_csv(path), after)



def featurize(ev, base, md, mg):
    F = monitor.Features(base, md, mg)
    out = pd.DataFrame([F.row(r._asdict()) for r in ev.itertuples(index=False)])
    for c in ("t", "type", "userId", "ip"):
        out[c] = ev[c].values if c != "t" else list(ev["t"])
    return out



def rule_flags(df):
    """Rule 1: >=5 LOGIN_ERROR from the same IP within the trailing 5 minutes. Rule 2: LOGIN from an IP outside
the allow-list."""
    df = df.reset_index(drop=True)
    r1 = np.zeros(len(df), bool)
    t = pd.to_datetime(pd.Series(list(df.t)), utc=True, format="ISO8601").reset_index(drop=True)
    for ip, g in df[df.type == "LOGIN_ERROR"].groupby("ip"):
        times = t[g.index].values
        for i, idx in enumerate(g.index):
            r1[df.index.get_loc(idx)] = (times[i] - times[: i + 1] < np.timedelta64(300, "s")).sum() >= 5
    r2 = ((df.type == "LOGIN") & ~df.ip.isin(ALLOWED)).values
    return r1, r2



def rules_grouped(df, key):
    """Apply the rules to each group on its own, so one run cannot leak into the next."""
    r1, r2 = np.zeros(len(df), bool), np.zeros(len(df), bool)
    for _, g in df.groupby(key, sort=False):
        a_, b_ = rule_flags(g)
        pos = df.index.get_indexer(g.index)
        r1[pos], r2[pos] = a_, b_
    return r1, r2



def flags_for(df, det, key=None):
    r1, r2 = rules_grouped(df, key) if key else rule_flags(df)
    m, lo, span, thr = det["ae"]
    out = {}

    def ae(d):
        e = catavo_ml.errors(m, (catavo_ml.prep(d) - lo) / span)
        return e > thr

    out["Autoencoder"] = ae(df)
    for name, col in (("AE, gap frozen", "login_gap"), ("AE, new_ip frozen", "new_ip"),
                      ("AE, failed-logins frozen", "failed_before_login")):
        d = df.copy()
        d[col] = det["med"][col]
        out[name] = ae(d)
    s, th = baseline_if.score_if(df)
    out["Isolation Forest"] = s > th
    out["Isolation Forest, 95th pct"] = s > det["if95"]
    out["Rule 1 (brute force)"], out["Rule 2 (unknown IP)"] = r1, r2
    out["Rules 1 OR 2"] = r1 | r2
    return pd.DataFrame(out, index=df.index)



def label_attacks(log, runs):
    log = log.copy()
    log["t"] = pd.to_datetime(log.event_time, utc=True, format="ISO8601")
    log = log.drop_duplicates(["event_time", "type", "userId", "ip"],
keep="last").sort_values("t").reset_index(drop=True)
    log["run_id"], log["scenario"], log["source"] = None, None, None
    for r in runs.itertuples():
        ip = HOST_IP.get(r.host)
        if ip is None:
            continue
```

```
        m = (log.ip == ip) & (log.t >= r.start - pd.Timedelta(seconds=2)) & (log.t <= r.end +
pd.Timedelta(seconds=15))
        log.loc[m, ["run_id", "scenario", "source"]] = [r.run_id, r.scenario, "Kali (new IP)" if
ip.endswith(".40") else "laptop (known IP)"]
    return log[log.run_id.notna()].reset_index(drop=True)



def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--normal2", help="optional fresh normal events (Splunk export) for an unbiased false-positive
rate")
    a = ap.parse_args()

    base = json.load(open(P("baseline_ips.json")))
    ft = pd.read_csv(P("features.csv"))
    med = {c: float(ft[c].median()) for c in FEATS}
    model, lo, span, thr = catavo_ml.load_model()
    det = {"ae": (model, lo, span, thr), "med": med}

    runs = pd.concat([pd.read_csv(P("attack_runs_kali.csv")), pd.read_csv(P("attack_runs_laptop.csv"))])
    runs["start"] = pd.to_datetime(runs.start, utc=True, format="ISO8601")
    runs["end"] = pd.to_datetime(runs.end, utc=True, format="ISO8601")
    atk = label_attacks(pd.read_csv(P("monitor_log.csv")), runs)
    meta0 = atk.drop_duplicates("run_id").set_index("run_id")[["scenario", "source"]]
    parts = []
    for rid, g in atk.groupby("run_id", sort=False):       # every run starts with fresh state (an isolated
incident)
        f = featurize(g[["t", "type", "userId", "ip"]].reset_index(drop=True), base, med["session_duration"],
med["login_gap"])
        f["run_id"] = rid
        parts.append(f)
    atk = pd.concat(parts, ignore_index=True)
    atk["scenario"], atk["source"] = atk.run_id.map(meta0.scenario), atk.run_id.map(meta0.source)
    atk_f = atk
    missing = sorted(set(runs.run_id) - set(atk.run_id))
    if missing:
        print(f"WARNING: {len(missing)} runs have no scored events (excluded): {missing[:5]}...")

    sp = pd.read_csv(P("keycloak_normal_split.csv"))
    pieces = []
    for b, g in sp.groupby("batch"):                       # each batch is featurized on its own (fresh state)
        ev = clean(g, keep=("part", "batch"))
        f_ = featurize(ev[["t", "type", "userId", "ip"]], base, med["session_duration"], med["login_gap"])
        f_["part"], f_["batch"] = ev.part.values, ev.batch.values
        pieces.append(f_)
    nrm_all = pd.concat(pieces, ignore_index=True)
    nrm = nrm_all[nrm_all.part == "train"].reset_index(drop=True)          # in-sample
    hold = nrm_all[nrm_all.part == "holdout"].reset_index(drop=True)       # never used for training
    det["if95"] = float(np.percentile(baseline_if.score_if(nrm)[0], 95))      # looser threshold, for comparison
    fa, fn, fh = flags_for(atk_f, det, key="run_id"), flags_for(nrm, det), flags_for(hold, det)
    dets = list(fa.columns)

    print(f"\nattack attempts scored: {len(atk)} in {atk.run_id.nunique()} runs;  normal attempts: {len(nrm)} in-
sample, {len(hold)} holdout")
    print("in-sample FPR by batch (evening B1 / daytime B2):")
    print(fn.groupby(nrm.batch.values).mean().mul(100).round(1)[["Autoencoder", "Isolation Forest"]].to_string())
    summ = pd.DataFrame({"in-sample FPR %": (fn.mean() * 100).round(1),
                         "holdout FPR %": (fh.mean() * 100).round(1),
                         "attempt recall %": (fa.mean() * 100).round(1)})
    fa["run_id"] = atk.run_id.values
    byrun = fa.groupby("run_id").any()
    summ["runs detected %"] = (byrun.mean() * 100).round(1)
    meta = atk.drop_duplicates("run_id").set_index("run_id")[["scenario", "source"]]
    sa, th_if = baseline_if.score_if(atk_f)
    sn, _ = baseline_if.score_if(nrm)
    ea = catavo_ml.errors(model, (catavo_ml.prep(atk_f) - lo) / span)
    en = catavo_ml.errors(model, (catavo_ml.prep(nrm) - lo) / span)
    print(f"\nscore check  AE threshold {thr:.4f}: normal median {np.median(en):.4f} max {en.max():.4f}; attack
median {np.median(ea):.4f}")
    print(f"                IF threshold {th_if:.4f}: normal median {np.median(sn):.4f} max {sn.max():.4f}; attack
median {np.median(sa):.4f} max {sa.max():.4f}")
    dbg = atk_f.assign(ae=ea, iff=sa)
    print("\nattack rows, median per scenario/source:")
    print(dbg.groupby(["scenario", "source"])[FEATS + ["ae", "iff"]].median().round(3).to_string())
    print("\nnormal rows with the highest IF score:")
    print(nrm.assign(iff=sn).nlargest(5, "iff")[FEATS + ["iff"]].round(3).to_string())
    print("\n=== Overall (rates in %) ===")
    print(summ.to_string())

    print("\n=== Runs detected, by scenario and source (detected/total) ===")
    j = byrun.join(meta)
    tab = {}
    for (sc, src), g in j.groupby(["scenario", "source"]):
        tab[f"{sc} | {src}"] = {d: f"{int(g[d].sum())}/{len(g)}" for d in dets}
    print(pd.DataFrame(tab).T.to_string())

    print("\n=== Attempts until first alert (median, over detected runs) ===")
    first = {}
    for d in dets:
        n = []
        for rid, g in fa.assign(t=atk.t.values).sort_values("t").groupby("run_id"):
            h = np.flatnonzero(g[d].values)
            if len(h):
                n.append(h[0] + 1)
        first[d] = float(np.median(n)) if n else None
    print(pd.Series(first).to_string())

    print("\n=== False positives by attempt type ===")
    print(fn.groupby(nrm.type.values).mean().mul(100).round(1)[["Autoencoder", "Isolation Forest", "Rule 1 (brute
force)", "Rule 2 (unknown IP)"]].to_string())

    summ.to_csv(P("eval_summary.csv"))
    byrun.join(meta).to_csv(P("eval_runs.csv"))
    print("\nsaved eval_summary.csv and eval_runs.csv in", DATA)

    if False and a.normal2:
        n2 = featurize(load_events(a.normal2), base, med["session_duration"], med["login_gap"])
        f2 = flags_for(n2, det)
```

```
        print(f"\n=== Fresh normal traffic ({len(n2)} attempts, not used in training) - false-positive rate %
===")
        print((f2.mean() * 100).round(1).to_string())


if __name__ == "__main__":
    main()
```

Appendix B: Complete Evaluation Output

Output of python evaluate.py | Tee-Object C:\catavo-ml\data\eval_output.txt for the final four-feature model, unchanged.

```
(venv) PS C:\catavo-ml> python evaluate.py | Tee-Object C:\catavo-ml\data\eval_output.txt
WARNING: 1 runs have no scored events (excluded): ['brute force-catavo-kali-1014015052-1']...


attack attempts scored: 420 in 54 runs;  normal attempts: 761 in-sample, 82 holdout
in-sample FPR by batch (evening B1 / daytime B2):
    Autoencoder  Isolation Forest
B1          0.2               0.2
B2          4.9               0.8


score check  AE threshold 0.0230: normal median 0.0007 max 0.0908; attack median 0.3117
             IF threshold 0.6628: normal median 0.4337 max 0.6829; attack median 0.6530 max 0.6536


attack rows, median per scenario/source:
                                  login hour  session duration  failed before login  new ip  login gap     ae    iff
scenario          source
brute force       Kali (new IP)       22.183            34.04                  7.0     1.0     48.072  0.692  0.654
                  laptop (known IP)   22.183            34.04                  7.0     0.0     48.072  0.457  0.654
slow low          Kali (new IP)       22.592            34.04                  2.5     1.0     48.072  0.300  0.630
                  laptop (known IP)   22.625            34.04                  2.5     0.0     48.072  0.003  0.630
spray             Kali (new IP)       22.300            34.04                  1.0     1.0     48.072  0.312  0.445
                  laptop (known IP)   22.317            34.04                  1.0     0.0     48.072  0.006  0.445
stolen credential Kali (new IP)       22.017            34.04                  0.0     1.0     48.072  0.312  0.405
                  laptop (known IP)   22.025            34.04                  0.0     0.0     48.072  0.000  0.405
token burst       Kali (new IP)       22.108            34.04                  0.0     1.0      2.104  0.376  0.653
                  laptop (known IP)   22.108            34.04                  0.0     0.0      2.296  0.067  0.653


normal rows with the highest IF score:
     login hour  session duration  failed before login  new ip  login gap    iff
751      10.117             34.04                    2       0     13.144  0.683
348      18.917             34.04                    0       0    998.638  0.670
747      10.117             34.04                    2       0     20.203  0.656
519      20.000             34.04                    3       0     47.072  0.653
698       9.800             34.04                    0       0      5.151  0.653


=== Overall (rates in %) ===
                          in-sample FPR %  holdout FPR %  attempt recall %  runs detected %
Autoencoder                           0.9            2.4              82.6            83.3
AE, gap frozen                        0.0            0.0              69.8            72.2
AE, new ip frozen                     0.9            2.4              57.1            48.1
AE, failed-logins frozen              0.9            1.2              67.1            72.2
Isolation Forest                      0.3            2.4               0.0             0.0
Isolation Forest, 95th pct            4.9            8.5              64.3            59.3
Rule 1 (brute force)                  0.0            0.0              37.6            48.1
Rule 2 (unknown IP)                   0.0            0.0              15.7            22.2
Rules 1 OR 2                          0.0            0.0              53.3            70.4


=== Runs detected, by scenario and source (detected/total) ===
                                  Autoencoder AE, gap frozen AE, new ip frozen AE, failed-logins frozen Isolation Forest Isolation Forest, 95th pct Rule 1 (brute force) Rule 2 (unknown IP) Rules 1 OR 2
brute force | Kali (new IP)             6/6            6/6               6/6                      6/6               0/6                        6/6                   6/6                     0/6           6/6
brute force | laptop (known IP)         6/6            6/6               6/6                      0/6               0/6                        6/6                   6/6                     0/6           6/6
slow low | Kali (new IP)                3/3            3/3               2/3                      3/3               0/3                        3/3                   2/3                     0/3           2/3
slow low | laptop (known IP)            0/3            0/3               0/3                      0/3               0/3                        3/3                   0/3                     0/3           0/3
spray | Kali (new IP)                   6/6            6/6               0/6                      6/6               0/6                        1/6                   6/6                     0/6           6/6
spray | laptop (known IP)               6/6            6/6               0/6                      6/6               0/6                        1/6                   6/6                     0/6           6/6
stolen credential | Kali (new IP)       6/6            6/6               0/6                      6/6               0/6                        0/6                   0/6                     6/6           6/6
stolen credential | laptop (known IP)   0/6            0/6               0/6                      0/6               0/6                        0/6                   0/6                     0/6           0/6
token burst | Kali (new IP)             6/6            6/6               6/6                      6/6               0/6                        6/6                   0/6                     6/6           6/6
token burst | laptop (known IP)         6/6            0/6               6/6                      6/6               0/6                        6/6                   0/6                     0/6           0/6

=== Attempts until first alert (median, over detected runs) ===
Autoencoder                1.0
AE, gap frozen             1.0
AE, new ip frozen          5.0
AE, failed-logins frozen   1.0
Isolation Forest           NaN
Isolation Forest, 95th pct 4.0
Rule 1 (brute force)       5.0
Rule 2 (unknown IP)        1.0
Rules 1 OR 2               5.0

=== False positives by attempt type ===
             Autoencoder  Isolation Forest  Rule 1 (brute force)  Rule 2 (unknown IP)
LOGIN                1.0               0.3                   0.0                  0.0
LOGIN ERROR          0.0               0.0                   0.0                  0.0


saved eval_summary.csv and eval runs.csv in C:\catavo-ml\data
(venv) PS C:\catavo-ml>
```

Appendix C: Splunk Searches Used

Table 27: Searches

Purpose	Search (SPL)
Keycloak events by type	index=keycloak | stats count by type
Login events table	index=keycloak (type=LOGIN OR type=LOGIN_ERROR) | table _time, type, username, ipAddress, clientId
Rule 1: brute force	index=keycloak type=LOGIN_ERROR | bin _time span=5m | stats count by _time, ipAddress | where count>=5
Rule 2: unknown source	index=keycloak type=LOGIN NOT ipAddress="192.168.100.50" NOT ipAddress="192.168.100.70" | stats count by ipAddress, userId
Windows logons	index=ad OR index=endpoint (EventCode=4624 OR EventCode=4625) | stats count by index, host, EventCode